

Grado en

Business Data Analytics

Informe final

Reto 08

Curso: 2º 2023-2024

Equipo: Azul Claro

URL del repositorio en GitHub: https://github.com/mikelbernabezorroza/reto_08.git

Ruta de la carpeta de Google Drive que contiene el proyecto:

https://drive.google.com/drive/u/0/folders/1_vfM-19ZA1YxjTV4SCZe6O48xEsVleo

Autores:

- Unai Álvarez de Ibarzabal
- Mikel Bernabé Zorroza
- Ane Fernández Quintana
- Guillermo Rubio Ramírez
- Lucía Villuela Cuenca
- Aitana Zas Vara

Índice

Índice	2
Índice de figuras	3
Índice de tablas	3
1.Introducción	4
2. Identificar la problemática	5
2.1. Sobre el cliente	5
2.2. Sobre la problemática	5
2.3. Objetivos	5
3. Recoger y almacenar los datos	6
3.1. Fuentes de datos utilizadas	6
3.2. Procesamiento de los datos	6
3.2.1. Organización técnica del proyecto	6
3.2.2. Data Discovery & Data Cleaning	7
4. Analizar y modelar los datos	9
4.1. Métricas tráfico de red	9
4.2. Modelar los datos	12
4.2.1. Detección de anomalías	12
4.2.2 Algoritmo Genético	21
4.3 Creación de la infraestructura	24
4.4. Visualizar los resultados y Conclusiones	25
4.4.1. Conclusiones anomalías	25
4.4.2. Conclusiones algoritmo genético	25
4.4.3. Conclusiones Infraestructura	26
4.5. Sistema publisher/subscriber	26
6. Implicaciones legales y éticas	28
Bibliografía	29
Apéndice I	31
Isolation Forest	31
One Class SVM	33
Kmeans	34
Apéndice II	38
Apéndice III	42
Apéndice IV	45
Glosarios	49
Glosario de siglas	49
Glosario de términos	49

Índice de figuras

Figura I - Diagrama de la organización del trabajo	4
Figura II - Gráfico del codo Apache	13
Figura III - Gráfico Silhouette Apache	13
Figura IV - Anomalías detectadas por IsolationForest en Apache	156*
Figura V - Distribución de las puntuaciones de anomalías por IsolationForest en Apache	15
Figura VI - Anomalías detectadas por OneClassSVM en Apache	16
Figura VII - Gráfico de dispersión http_status y response_size por anomalías	16
Figura VIII - Gráfico de correlaciones	17
Figura IX - Gráfico del codo Firewall	17
Figura X - Gráfico Silhouette Firewall	18
Figura XI - Anomalías detectadas por IsolationForest en Firewall	19
Figura XII - Distribución de las puntuaciones de anomalías por IsolationForest en Firewall	20
Figura XIII - Anomalías detectadas por OneClassSVM en Firewall	20
Figura XIV - Gráfico de dispersión Bytes y Elapsed Time por anomalías	20
Figura XV - Gráfico de líneas de incremento en la función fitness	22
Figura XVI - Diagrama del sistema publisher/subscriber	27

Índice de tablas

Tabla I - Configuración de la Máquina Virtual Colectiva	6
Tabla II - Descripción de las métricas por cluster	14
Tabla III - Caracterización por cluster	18
Tabla IV - Mejores parámetros para el algoritmo genético probando todas las combinaciones posibles	23
Tabla V - Mejores parámetros para el algoritmo genético probandolos individualmente	23

1.Introducción

El objetivo principal de este reto es el **diseño y despliegue de una arquitectura de Big Data** que permita la ingesta, manipulación, análisis y visualización de datos en tiempo real provenientes de dispositivos de una planta industrial. Las herramientas centrales de esta arquitectura es el **stack de Elastic**, que incluye Logstash, Elasticsearch y Kibana.

Asimismo, se ha desarrollado una infraestructura para combinar datos de tráfico de red como resultados operacionales de la misma industria -en este caso, sensórica de los compresores-. Los datos se ingieren automáticamente y en tiempo real mediante **Logstash**, que los envía a **Elasticsearch** para su almacenamiento y análisis. **Kibana** se utiliza para visualizar en un dashboard parámetros relevantes y proporcionar una visión clara y accesible de los datos, permitiendo una monitorización efectiva.

Por otro lado, la empresa industrial cuenta con compresores equipados con sensores que estiman la frecuencia de operación de cuatro de sus compresores. Estos dispositivos regulan el volumen de aire necesario para mantener condiciones óptimas para los empleados. Para ello, se ha desarrollado un **algoritmo genético** para maximizar la eficiencia energética de estos. Los resultados del algoritmo se guardan en una base de datos de **MongoDB** y finalmente se visualizan en un cuadro de mandos en **Dash**, facilitando la toma de decisiones para la optimización del uso de energía.

En este proceso, se utiliza el **broker MQTT de Mosquitto** como intermediario para realizar consultas sobre los resultados de los compresores. Estos datos se indexan en Elasticsearch, permitiendo su visualización en tiempo real a través de un dashboard en Kibana. Esta integración permite una monitorización continua y un análisis detallado de los datos operacionales, mejorando la capacidad de respuesta ante posibles anomalías y optimizando el rendimiento de los compresores.

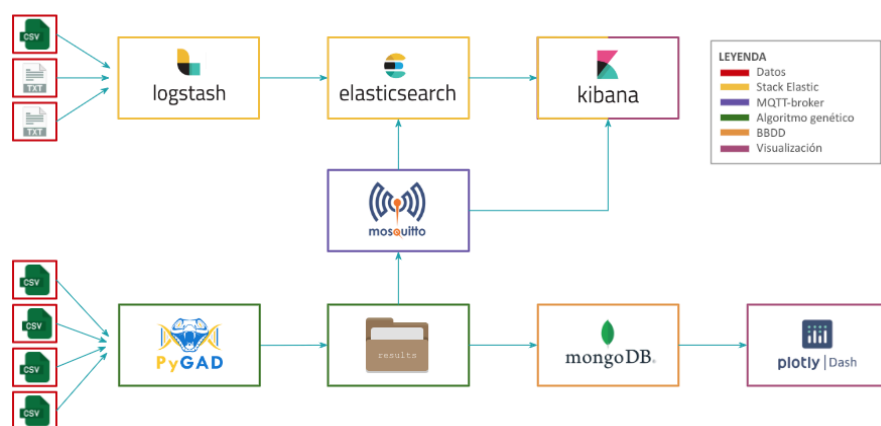


Figura I - Diagrama de la organización del trabajo

2. Identificar la problemática

2.1. Sobre el cliente

LANIT es un **grupo empresarial** con más de 25 años de experiencia en consultoría tecnológica y empresarial, destacándose en **analítica de datos y mantenimiento integral de infraestructuras tecnológicas**. Su crecimiento sólido se ve respaldado por una continua evolución en la adopción de aplicaciones y metodologías ágiles para abordar los desafíos diarios de sus clientes (Grupo Lanit, 2022). Una de sus principales entidades es **LANIT Consulting**, siendo el cliente directo que ha propuesto el reto. Esta sociedad formada por más de 40 profesionales altamente capacitados en ingeniería informática y matemáticas, ofrece soluciones innovadoras a través de la analítica, y gestionan proyectos llave en mano o colaboran con los equipos del cliente, ya sea en sitio o de forma remota.

2.2. Sobre la problemática

Una planta industrial se enfrenta a una dificultad en su proceso de producción debido a la **falta de observabilidad en tiempo real**. La ausencia de una infraestructura Big Data impide **monitorear** adecuadamente el comportamiento de los dispositivos, lo que aumenta el riesgo de **fallos inesperados, reducción de la eficiencia y calidad del producto**, así como posibles peligros para la seguridad de los trabajadores y el medio ambiente. La implementación de un sistema de observabilidad se vuelve urgente para identificar y abordar problemas potenciales antes de que se conviertan en crisis operativas.

2.3. Objetivos

Este reto incluye varios objetivos, los cuales tienen que ser específicos y medibles para su perfecta ejecución. Todos los objetos tendrán un plazo máximo de 6 semanas con revisiones y ajustes periódicos cada 2 semanas. A continuación, se describen dichos objetivos:

1. **Diseño y despliegue de una arquitectura de Big Data** que permita la ingesta, manipulación, análisis y visualización de datos en tiempo real provenientes de dispositivos de una planta industrial utilizando el stack de Elastic.
2. **Optimización de la eficiencia energética** de cuatro compresores industriales a través del desarrollo e implementación de un algoritmo genético.
3. **Almacenar los resultados** en la base de datos de MongoDB, **visualizando** el rendimiento y las recomendaciones en un cuadro de mandos en Dash.
4. **Realizar consultas** sobre los datos de los compresores a través del broker MQTT de Mosquitto y **visualizarlos** en tiempo real en un dashboard de Kibana.

3. Recoger y almacenar los datos

3.1. Fuentes de datos utilizadas

Con el objetivo de instaurar una infraestructura de Big Data destinada a la observación en tiempo real de los dispositivos en una planta industrial, se han empleado **dos fuentes de datos** distintas: datos de tráfico de red y datos provenientes de cuatro compresores de aire. La información de **tráfico de red abarca tres conjuntos de datos**, que incluyen registros de Firewall, traducidos a un log en formato csv, SNMP Wifi, entregado como archivo de texto, y Network Flows, proporcionado como registros de Apache en formato de texto. En cuanto a los **datos de los compresores**, se ha proporcionado **un fichero de datos para cada compresor** de aire en formato csv.

3.2. Procesamiento de los datos

3.2.1. Organización técnica del proyecto

Para establecer una infraestructura de Big Data con el stack de Elastic, se requiere un sistema operativo que sea compatible con esta tecnología. Por consiguiente, se ha optado por emplear una **máquina virtual**, en nuestro caso VirtualBox, para garantizar un entorno de trabajo uniforme para todos los colaboradores del proyecto. Esta máquina se ha sido configurada con las siguientes especificaciones:

Tipo SO de la máquina	
Tipo SO invitado	Ubuntu 22.04 LTS (Jammy Jellyfish) (64-bit)
Hardware	
Memoria base	4096
Procesadores	4
Habilitar EFI	False
Disco	
Tamaño de disco	50,00 GB
Reservar tamaño completo	True

Tabla I - Configuración de la Máquina Virtual Colectiva

Tras la configuración inicial de la máquina, se ha procedido a la **instalación de todas las herramientas** requeridas para la ejecución del proyecto, estas incluyen, Elasticsearch y Kibana del stack de Elastic, junto con el broker Mosquitto, todas implementadas mediante comandos en la terminal. Además, se ha incorporado VSCode como editor de código.

Simultáneamente a la configuración de la máquina virtual, se ha establecido un **repositorio en GitHub** con el propósito de facilitar la colaboración entre los participantes del proyecto al

centralizar la gestión del código y rastrear los cambios mediante el control de versiones. Hecho esto, se ha definido la estructura de directorios que seguirá el proyecto.

Una vez completado este proceso, se ha procedido a la instalación de `Anaconda` con el objetivo de **crear un entorno** que asegure la coherencia de las versiones de las librerías, evitando así posibles conflictos. Además, se ha elaborado un archivo `README` para **documentar el proyecto y suministrar instrucciones** de instalación de las bibliotecas necesarias. Posteriormente, se ha generado un archivo `.gitignore` para **excluir los datos** de trabajo del repositorio. Por último, se ha de destacar que **Python** ha sido el lenguaje de programación utilizado para el desarrollo de este proyecto.

3.2.2. Data Discovery & Data Cleaning

Tan pronto como se ha configurado un entorno adecuado para el trabajo grupal, se ha llevado a cabo la carga de datos en **Visual Studio Code** para su posterior limpieza. Hecho esto, se ha realizado un análisis preliminar de los datasets con el objetivo de obtener una comprensión general de los mismos, incluyendo una descripción de las variables y cambios de formatos, si lo necesitasen, entre otros aspectos. Además, se ha realizado un estudio estadístico para las variables cuantitativas y se han elaborado tablas de frecuencia para las variables cualitativas.

En el dataset “Network Flows”, se han identificado **8144 filas duplicadas**, equivalente al 0.23% del tamaño del archivo original. Alrededor del 83% de las duplicidades provienen de solo cuatro direcciones IP de origen, mientras que las de destino han mostrado una distribución más equitativa, aunque una de ellas ha destacado al concentrar la mayor parte de los duplicados. Además, se han detectado **combinaciones específicas de ID Flow** que acumulaban la mayoría de las duplicidades, algunas de las cuales tenían una duración breve, mientras que otras mostraron una duración más prolongada. En cuanto a la temporalidad, aproximadamente el 83% de las duplicidades se han concentrado en dos franjas horarias específicas, de 4 am a 5 am y de 9 am a 12 pm, con un descenso a las 11 am. Por último, se destaca un **mayor porcentaje de duplicados en el mes de mayo** en comparación con abril.

Por otro lado, en el conjunto de datos del Firewall se han identificado **8362 instancias duplicadas**, representando un 12,76% del total de los datos. Más del 13% de estas duplicidades provenían de tres puertos de origen específicos, mientras que aproximadamente el 30,5% de los datos duplicados provenían de otros tres puertos de destino. Estos hallazgos contrastan con la distribución de los puertos de origen y destino NAT, donde un solo puerto representaba la mayoría de los duplicados. Además, un puerto destaca por ser el receptor y emisor de la mayor cantidad de bytes simultáneamente. Asimismo, es el que recibe y envía la mayor cantidad de iteraciones de paquetes, con un tiempo medio de conexión de 196 segundos.

Por último, en el archivo de registro de Apache, se han identificado un total de **2577 filas duplicadas**, en otras palabras, el 19% del fichero original son duplicidades, y se ha detectado la duplicación de una columna, aunque esta variable no proporcionó información relevante. En cuanto a las direcciones IP de los clientes, se ha encontrado sólo dos duplicadas, una de las cuales fue responsable de la mayoría de los duplicados. Ambas direcciones IP de cliente están asociadas a un **navegador Mozilla** con sistema operativo Windows. En cuanto a los métodos HTTP utilizados, la mayoría fueron de tipo POST y GET, aunque también se han empleado otros métodos menos comunes. La respuesta de solicitud más frecuente fue la 200 o la 302 y 303, indicando una **solicitud exitosa o una redirección temporal**. La ruta de solicitud más común es el inicio de sesión, y la versión HTTP predominante es la 1.1 en todos los duplicados.

Después de analizar minuciosamente toda la información y al constatar que su presencia no añadía valor adicional y simplemente aumentaba la carga de los archivos, se ha optado por **eliminarla**, conservando únicamente la primera instancia. Como se ha mencionado previamente, se considera poco probable que en un intervalo de tiempo tan breve como un milisegundo puedan generarse cantidades tan significativas de instancias idénticas.

Volviendo a la limpieza general, se ha realizado un estudio de **valores ausentes** tanto en variables numéricas como categóricas. En primer lugar, se ha empleado una herramienta de visualización para mapear los datos faltantes en una estructura matricial por cada dataset, y no se ha detectado missing alguno. Hecho esto, se ha llevado a cabo un conteo que confirmó nuestra hipótesis de que no había ningún valor ausente.

Por último, se han evaluado los **valores atípicos y anómalos** utilizando, a priori, diagramas de caja y bigotes (boxplots). Al visualizar la distribución de estos valores, algunos datos no parecían coherentes. Por ello, se ha realizado un estudio basado en métricas generales de los archivos y, posteriormente, se han empleado modelos de Machine Learning para decidir si estos datos debían eliminarse -al considerarse anómalos- o mantenerse.

4. Analizar y modelar los datos

4.1. Métricas tráfico de red

Tras la finalización del procesamiento de datos, que ha incluido el análisis de duplicados y la detección de anomalías, se han establecido **grupos de datos en cuarentena**. Este estudio adicional tiene como objetivo abordar cualquier interrogante pendiente y obtener una comprensión más clara de los hallazgos obtenidos.

El registro del firewall recopila información de un dispositivo de seguridad de red que **monitorea y gestiona el flujo de datos entrantes y salientes** según un conjunto de reglas de seguridad predefinidas. Estos dispositivos son comúnmente utilizados en entornos domésticos y empresariales para agregar una capa adicional de **protección contra ciberataques** y otras amenazas a la red. Con el fin de evaluar el volumen de tráfico y la carga en la red, se ha realizado un análisis cuantitativo mediante el recuento de paquetes y bytes transmitidos. Los resultados muestran que se han transmitido más de **6.5 millones de paquetes**, equivalente a casi **6.634 millones de bytes**. Sin embargo, no ha sido posible realizar un desglose temporal debido a la falta de información.

Asimismo, se ha realizado un minucioso análisis de las **diferencias entre los bytes enviados y recibidos** en la red. De acuerdo con los resultados, se ha identificado que en el 35,2% de las interacciones, se han recibido más bytes de los que se han enviado, lo que puede indicar una variedad de actividades que incluyen: descargas, respuestas a solicitudes externas, tráfico de retorno y transferencia de datos entrantes hacia la red. Sin embargo, en el 64,8% restante del tráfico, se han enviado más bytes de los que se han recibido. Esta discrepancia en el volumen de bytes transmitidos y recibidos puede deberse a que el cliente esté utilizando el método HTTP POST, que se emplea para enviar datos al servidor con el fin de crear o actualizar un recurso. No obstante, al no tener dicha información, no se ha podido verificar.

Se ha llevado a cabo un análisis similar en lo que respecta a los **paquetes enviados y recibidos**, dando como resultado una tendencia constante. Aunque las proporciones puedan variar, el resultado principal es que la mayoría de las comunicaciones presentan un mayor número de paquetes enviados que recibidos. Concretamente, se ha observado que en el 86,3% de las interacciones, se han enviado más paquetes que los recibidos, mientras que en el 13,7% restante, se ha registrado lo contrario: más paquetes recibidos que enviados.

También se ha realizado un estudio sobre la **aceptación del firewall**, es decir, cómo ha respondido a las diversas solicitudes de comunicación, incluyendo las aceptadas, caídas, negadas, restablecidas, entre otras. Se destaca que, de cada 10 solicitudes, aproximadamente 6.5 han sido aceptadas sin inconvenientes, procedentes de los puertos de origen principales -no ha sido posible determinar el nombre del servicio para estos puertos- y desde el servicio

asociado NTP, con destino a los puertos principales y los asociados a los servicios NPT y de dominios privados, también han sido aceptadas. En cuanto a las acciones de tipo “Drop”, se ha registrado que **más del 20% de las comunicaciones han sido descartadas**, provenientes de los puertos de origen comunes y el puerto asociado al servicio HTTPS, con destino a los puertos principales.

En menor proporción, aproximadamente un 14% de las acciones, corresponden a solicitudes denegadas, y se han realizado desde los puertos de origen principales hasta el puerto de destino 445. Es relevante destacar que todas las conexiones de los usuarios que intentaron establecer comunicación con este puerto han sido denegadas. Además, se observa que estos puertos de origen han intentado establecer dicha comunicación en múltiples ocasiones, pero el firewall ha persistido en su negativa. Por último, se ha detectado la acción de restablecimiento en un 0.09% de las comunicaciones, desde los puertos principales hacia otros puertos principales. Es importante enfatizar que la frecuencia de estas interacciones es muy baja, limitándose a una sola ocasión.

Asimismo se ha analizado la **duración de las conexiones** establecidas. En total, se han registrado más de 57 mil conexiones, con una duración promedio de poco más de 75 segundos, lo que equivale a aproximadamente 1.25 minutos, lo que indica que las comunicaciones tienden a ser relativamente breves en promedio. Solo el 25% de todas ellas superan los 31 segundos de duración. A pesar de que no se ha podido identificar los nombres de los servicios asociados a estos puertos, se sugiere que podrían estar relacionados con consultas breves. Sin embargo, se observa una desviación estándar de más de 5 minutos, lo que muestra la presencia de valores atípicos o outliers, principalmente por encima de la media. Algunas conexiones llegan a extenderse hasta el máximo de 180 minutos, equivalente a tres horas. Aunque los nombres de los servicios asociados a estos puertos no se ha podido identificar, se especula que podrían estar relacionados con plataformas de películas o series, dada su duración.

Además, se han examinado los **puertos más comunes** tanto en origen como en destino dentro del firewall. Los puertos de origen más frecuentes son [27005, 57470, 49418, 6881, 443]. Aunque el nombre del servicio para la mayoría de estos puertos no ha podido identificarse, se sabe que el último puerto pertenece al servicio HTTPS. Por otro lado, los puertos de destino más recurrentes son [53, 443, 445, 80, 25174], asociados respectivamente a los servicios de dominios privados, HTTPS, Microsoft-DS, HTTP, aunque el nombre del servicio para el último puerto no ha sido determinado.

Finalmente, se ha explorado la **relación entre los puertos**, considerando aquellos con mayor tráfico en términos de volumen y número de conexiones. La combinación de puerto de origen y destino que registra el mayor volumen de bytes y paquetes corresponde al puerto de origen 57235 con destino al puerto 15187, con un total de 1,269 millones de bytes y más de un millón de paquetes, seguido por el puerto 15792 con destino al puerto 3478. Por otro lado, los puertos con el menor tráfico en volumen van desde el puerto 80 hacia los puertos de final 9918 y 18811, con un total de 60 bytes y un paquete. En cuanto a las conexiones, los puertos menos activos son la combinación entre el puerto 5 hasta el puerto 47993, con 62 bytes y 1 paquete, y desde el puerto 20 hasta el puerto 17095, con 554 bytes y un paquete también.

Por otro lado, se han estudiado las métricas del fichero de Apache. Este archivo proporciona detalles sobre cada solicitud realizada al servidor. En primer lugar, se ha llevado a cabo un **recuento de los diferentes métodos de solicitud** presentes en el archivo, y se destaca que, el método más utilizado es el POST, representando un 45.02% de las solicitudes, lo que equivale a 4973 comunicaciones al servidor para que acepte los datos de la solicitud. Le sigue muy de cerca el método GET, con un 37.04%, el cual se utiliza para solicitar datos de un recurso específico en un servidor web. Menos participación tiene el método HEAD, con un 17.56%, que es similar al método GET pero difiere en cómo maneja la respuesta.

Por otro lado, los métodos OPTIONS, PROFIND y NETSPARKER, ordenados descendientemente, tienen una participación mínima, no llegando ni al 0.5% de la suma de los tres. Estas solicitudes son menos frecuentes que las mencionadas anteriormente y tienen diversas funciones, como obtener información sobre las opciones de comunicación disponibles en un servidor, recuperar propiedades de recursos en servidores WebDAV, y solicitudes realizadas por la herramienta de seguridad Netsparker durante un escaneo de seguridad en una aplicación web.

Además, al anterior análisis se ha **incluido la variable temporal** tanto en los estados como en los métodos. Es importante señalar que los datos abarcan dos años distintos, concentrándose exclusivamente en los meses de diciembre, específicamente los días 22, en otras palabras, el rango de temporal se ciñe a dos días. Es por ello que se ha llevado a cabo una comparación entre ambos años, ya que el volumen de datos entre ambos años es similar, esto es, el 51.31% corresponde al año 2022 y el 48.69% al 2023.

En cuanto a los estados HTTP, en 2022 se destacan varios aspectos. El primero es la alta concentración de estados 303, que representan más de la mitad de los estados registrados ese año. Le sigue el estado 200, que indica conexiones correctas realizadas sin problemas, como era de esperar. Además, se han registrado otros estados menos comunes en rangos coherentes. Sin embargo, destaca que en 2023, el 46.26% de los datos corresponden al estado 404, lo cual es una proporción muy alta e incoherente para un “Not Found”. Le sigue de cerca

el estado 200, como era de esperar y en menor proporción, otros estados en porcentajes más lógicos.

Respecto a los métodos HTTP, en el año 2022, solo se han registrado **dos tipos de métodos**, GET y OPTIONS. Es relevante señalar que, el segundo método acumula casi el 83% de las solicitudes de ese año. En comparación con el 2023, este valor se incrementa más de 16 veces y llega a representar más del 94% de las solicitudes del archivo. Por otro lado, resulta extraño que, a pesar de ser GET uno de los métodos más frecuentes en el archivo como conjunto, tenga un valor tan bajo en este año.

4.2. Modelar los datos

4.2.1. Detección de anomalías

Como se ha mencionado anteriormente, se han obtenido métricas de los archivos de Firewall y Apache. En este primer análisis de las relaciones entre variables de los archivos, se han detectado relaciones inusuales, por lo que primero se ha procedido a inspeccionar dichos datos por **agrupaciones**. Dentro de una de estas, se han aplicado filtros basados en la temporalidad, la dirección IP del cliente y el estado de la conexión. Asimismo, se ha observado la presencia de valores anómalos, donde se registran casi 350 solicitudes desde el mismo cliente, misma IP, con respuestas consistentemente “Not Found”, todo en cuestión de milisegundos. Asimismo, es importante destacar que este descubrimiento no se trata de duplicados, ya que se han filtrado específicamente por esas variables. Además, es importante tener en cuenta que estas instancias originalmente contenían más variables que las distinguían de las demás.

Tras no detectar más comportamientos inusuales, se ha decidido aplicar dos técnicas de detección de anomalías. En primer lugar, se ha utilizado una técnica de clustering, concretamente el algoritmo **K-means**, junto con la métrica Silhouette para evaluar la coherencia de los grupos generados. Por otro lado, se han empleado el **Isolation Forest** y **One Class SVM** para entrenar un modelo adicional y realizar su evaluación.

En relación al archivo de Apache, inicialmente, se ha aplicado el modelo de **K-means**. Para ello, en primer lugar, se han **escalado los datos** para asegurar un funcionamiento óptimo del algoritmo al tener todas las variables en la misma escala. Tras esto, se han creado **variables dummy** para representar las columnas categóricas y se ha aplicado el **método del codo** para determinar el número óptimo de clusters, buscando así un equilibrio entre la compacidad de los clusters y la simplicidad del modelo. En un principio, el método del codo sugirió que el número óptimo de clusters era $k = 6$, por lo tanto, se ha creado el modelo de K-means con ese parámetro.

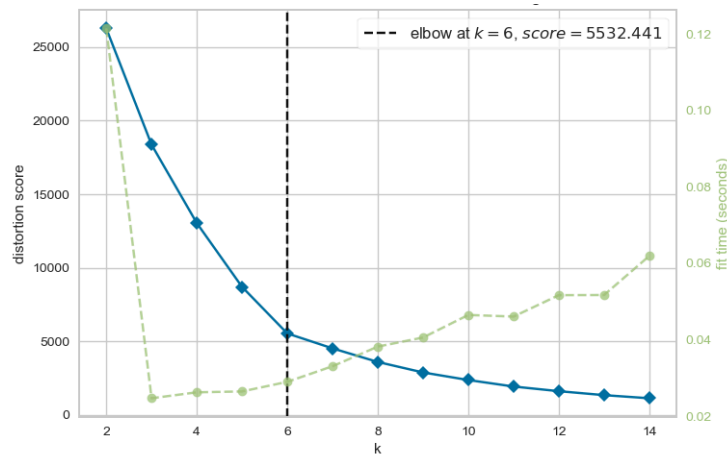


Figura II - Gráfico del codo Apache

Una vez construido el modelo de agrupación, se ha empleado el coeficiente de **Silhouette** con múltiples centroides. Esta métrica se ha utilizado principalmente para profundizar en la estructura de los datos, y así tomar decisiones más informadas sobre el cluster óptimo. El coeficiente de Silhouette permite evaluar la cohesión y la separación de los clusters, lo que ayuda a entender la distribución de los datos dentro de cada grupo y la separación entre sí. Esta evaluación es importante ya que asegura que los clusters sean significativos y representen de manera efectiva los patrones subyacentes en los datos.

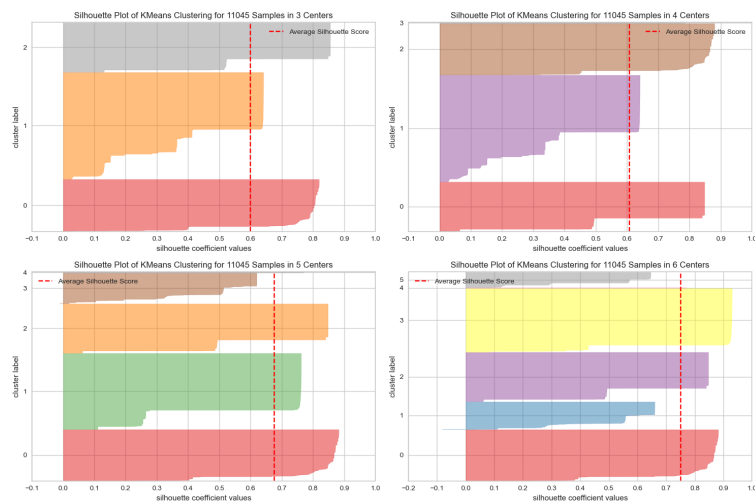


Figura III - Gráfico Silhouette Apache

Después de analizar ambos métodos, se ha decidido mantener el parámetro k obtenido por el método del codo. Esto se debe a que el índice de silueta ha mostrado buenos resultados, indicando pocas instancias mal clasificadas. Con esto definido, se ha realizado un estudio de cada cluster utilizando las métricas previamente obtenidas. A continuación, se presenta una tabla resumen para cada agrupación:

	Cluster 0	Cluster 1	Cluster 2	Cluster 3	Cluster 4	Cluster 5
Instancias	2726	1470	2622	3390	32	805
Método	GET	POST	HEAD	POST	GET	POST
Status	200	303	404	200	200	200
Tamaño respuesta media	1076.37	605.65	235.95	2839.64	22895.75	1173.85
Año	2023	2022	2023	2022	2023	2023

Tabla II - Descripción de las métricas por cluster

Vistos estos resultados, no se podría asignar ninguno de los cluster como tal anómalos. No obstante, la segunda y cuarta agrupación tienen algunas **características que difieren del resto**. Respecto al cluster 2, tiene un número significativo de instancias, su método HTTP predominante es HEAD, lo cual es inusual, observando las métricas tanto de las demás agrupaciones como del archivo sin clusters, y el estado HTTP que destaca es el 404, lo que indica que podría estar manejando una cantidad inusualmente alta de solicitudes que resultan en errores de página no encontrada. Respecto al cluster 4, tiene un número muy bajo de instancias en comparación con las otras agrupaciones, el tamaño medio de respuesta es mucho mayor que en los otros clusters, lo que podría estar representando un tipo específico de solicitud que devuelva grandes cantidades de datos, lo cual podría ser inusual en comparación con el resto de los clusters.

Estos clusters requieren una mayor atención para entender por qué difieren de los demás y si estas diferencias indican la existencia de anomalías o comportamientos inusuales en los datos. Sin embargo, la información proporcionada no es suficiente como para considerar alguno de estos grupos como anómalos, ya que es probable que la mayoría de estas instancias no sean inusuales. Como conclusión, sería necesario **identificar agrupaciones de comportamientos** mediante la agrupación de varias variables, como se ha mencionado anteriormente.

Como se ha mencionado anteriormente, para la detección de **valores anómalos** también se ha decidido optar por la creación de los modelos **Isolation Forest y One Class SVM** tanto para dataframe Apache como para el de Firewall. En primer lugar se ha entrenado el modelo Isolation Forest, que consiste en generar árboles de decisión aleatorios y los valores con menos profundidad en el árbol los considera como anomalía. El modelo One Class SVM en cambio, en el entrenamiento aprende una función que separa los datos normales de los anómalos, y al realizar las predicciones, los valores que caigan fuera de ese rango los interpreta como anomalía. Estos dos modelos como resultado de la predicción asignan -1 a las instancias clasificadas como anómalas y 1 a las clasificadas como normales.

Para el dataset **Apache** se ha creado en primer lugar un modelo IsolationForest. Este modelo se ha entrenado y predicho con el dataset utilizado para crear el modelo de K-means. Además, se ha añadido el parámetro contaminación = 0,2 para conseguir un 20% de anomalías. Los valores predichos como normales y anómalos son los siguientes.

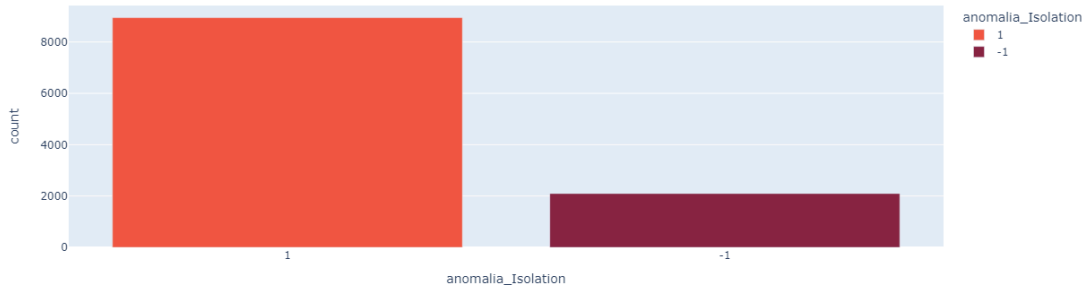


Figura IV - Anomalías detectadas por IsolationForest en Apache

Isolation Forest asigna ciertas puntuaciones a cada observación dependiendo del grado de anomalía que sea. En el siguiente gráfico se observan dichas puntuaciones en el dataset Apache. En este caso, la mayoría de los datos se encuentran con una puntuación mayor a 0 lo que indica que estas instancias son las más normales. Los valores cuya puntuación es negativa indican que el nivel de anomalía es grande.

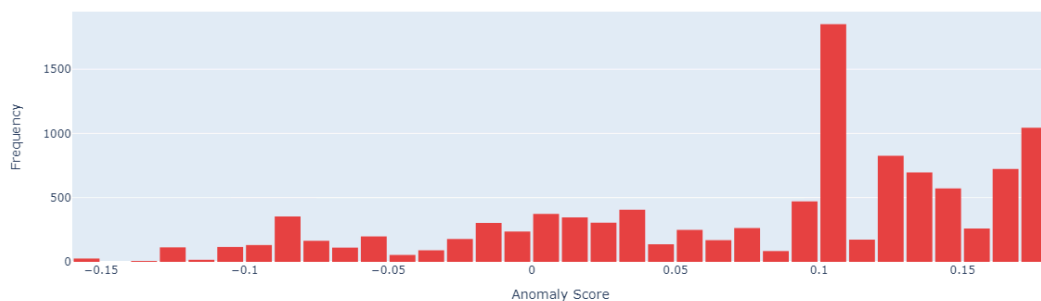


Figura V - Distribución de las puntuaciones de anomalías por IsolationForest en Apache

Una vez obtenidas las posibles instancias anómalas y normales predichas por Isolation Forest, se ha entrenado un modelo de One Class SVM con los datos normales y se han predicho los anómalos con el fin de lograr mejores resultados. Este segundo modelo, ha caracterizado como anomalías 1738 de las 2096 que había predicho Isolation Forest.

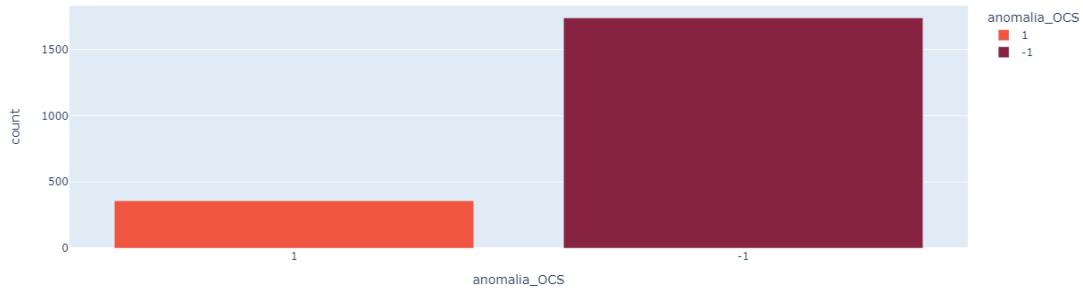


Figura VI - Anomalías detectadas por OneClassSVM en Apache

Finalmente, tras detectar las posibles instancias normales y anómalas, se han concatenado los dataset y se ha añadido una columna llamada “anomalías” que contiene los valores -1 y 1. Estos valores representan respectivamente una instancia anormal y una instancia normal.

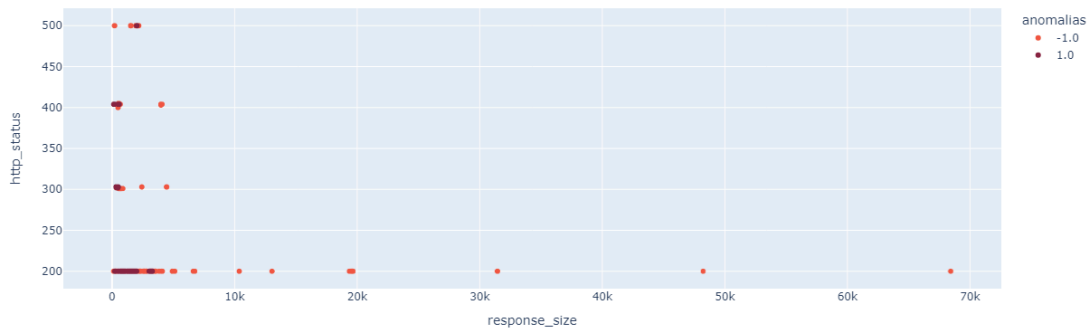


Figura VII - Gráfico de dispersión http_status y response_size por anomalías

En este gráfico se puede observar la dispersión que hay entre la variable “http_status” y “response_size”, los colores están acorde la predicción de los modelos, es decir anomalía o normal. Los valores detectados son como anómalos son los que tienen un response size de mucha cantidad. También hay ciertas observaciones predichas como anómalas en algún http_status, estos valores pueden deberse a alguna característica que tengan las instancias.

En cuanto al archivo Firewall, los pasos llevados a cabo para la detección de anomalías han sido parecidos a los de dataset de Apache. En primer lugar, antes de insertar los datos en el modelo se ha visto si alguno podría ser anómalo. Se ha observado si había alguna instancia cuya suma entre “Bytes Sent” y “Bytes Received” no tuviese el mismo resultado que “Bytes”, debido a que sería una anomalía, en este caso ninguna instancia lo cumplía. Con las variables “pkts_received”, “pkts_send” y “Packets” se ha calculado de la misma manera y tampoco había ninguna instancia que lo cumpliera. Después, se ha calculado la cantidad de datos que en la columna “Action” tenían el atributo “allow”, y a la vez, la columna “Elapsed Time (sec)” era igual a 0, ya que si cumplían esta condición puede deberse a un comportamiento automatizado

o malicioso que podría representar una amenaza para la seguridad del sistema. En este caso, se han encontrado un total de 312 instancias con esta característica y han sido categorizadas como anómalas.

A continuación, se ha creado un “heatmap” que muestra la correlación que existe entre las diferentes variables numéricas del dataset. Se ha podido observar que entre las variables “Bytes”, “Packets” y “pkts_sent” hay una correlación positiva elevada, más de 0,8. Es por ello que, a la hora de crear el modelo, se ha seleccionado únicamente una de ellas, debido a que podría añadir cierto ruido a los resultados del modelo. Una vez seleccionadas las columnas a insertar, se ha procedido a escalar el dataset entero, ya que K-means utiliza la **distancia euclidiana** para determinar los clusters.

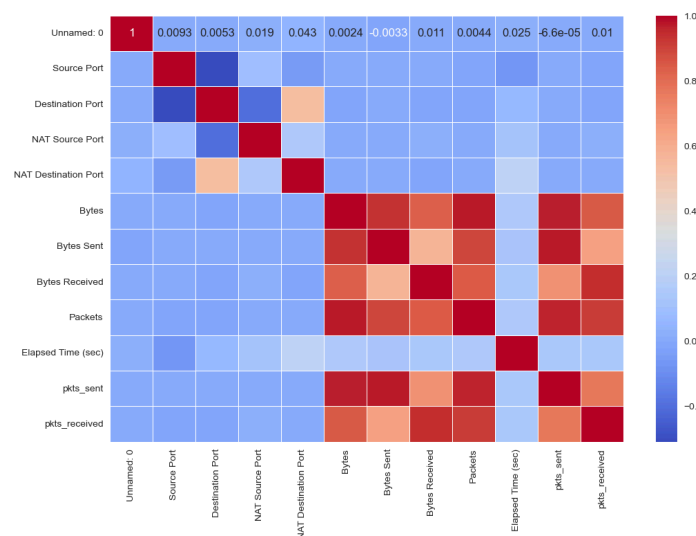


Figura VIII - Gráfico de correlaciones

Una vez preprocesados los datos, se ha observado mediante el **método del codo** el mejor valor de k para insertar en el modelo, y como se puede observar el valor óptimo ha sido 7.

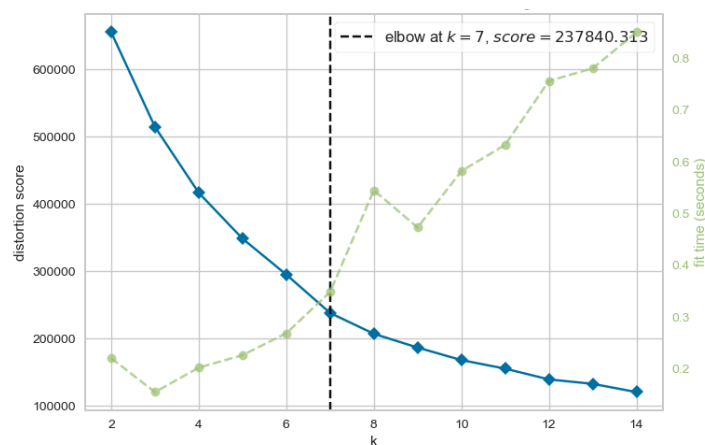


Figura IX - Gráfico del codo Firewall

Se ha entrenado el modelo con $k = 7$ y con el fin de saber si la agrupación era la correcta, y se ha observado que el **gráfico Silhouette** con valores de k entre 4 y 7. Mediante este gráfico se puede observar que la distribución que presentan los clusters no son muy correctas, pero el modelo que mejor **Coefficiente de Silhouette** tiene es cuando $k = 7$, por ello se ha optado por crear el modelo con este valor.

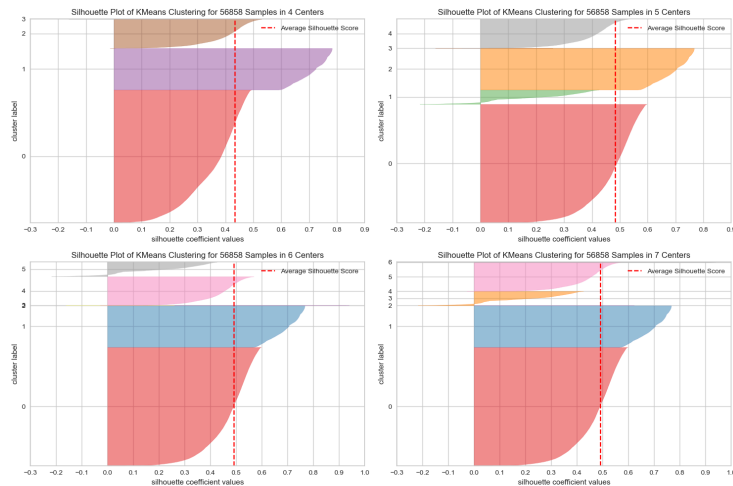


Figura X - Gráfico Silhouette Firewall

Una vez obtenidos los clústeres y sus instancias, se ha procedido a caracterizar cada uno de ellos:

	Cluster 0	Cluster I	Cluster II	Cluster III	Cluster IV	Cluster V	Cluster VI
Instancias	33112	11635	3	3998	1	8055	54
Bytes	Recibe muchos			Manda bastantes			Recibe bastantes
Acción mayoritaria	Allow	Deny		Deny	Drop	Allow	
Puerto Source común	57470	443	57235	51244	53294	27005	63996
Puerto Destino común	53	25174	15187	25174	445	27015	443
Tiempo transcurrido		Media = 0	Muy alto		Valor más alto		
Paquetes comportamiento	Similar			Similar			Similar

Tabla III - Caracterización por cluster

Observando los resultados obtenidos se pueden sacar ciertas conclusiones. El cluster 0 recibe muchos bytes y la mayoría de acciones son aceptadas, por ello no son anomalías. En cuanto al cluster I, la mayoría de las acciones son denegadas, además la media de tiempo transcurrido es 0, por ello este cluster podría ser detectado como anomalía. El cluster II es una clara anomalía debido a que contiene únicamente 3 instancias y sus tiempos transcurridos son muy altos. En cuanto al cluster III, la mayoría de acciones son denegadas, además los bytes que se mandan son bastante altos, por ello podría detectarse como anomalía. El cluster IV contiene una única instancia cuya acción es denegada y el tiempo transcurrido es el mayor del dataframe, por ello es una anomalía clara. El penúltimo cluster, es decir el V, a simple vista no tiene ningún atributo raro, además la mayoría de las peticiones son aceptadas. Finalmente, en cuanto al cluster VI, los bytes recibidos son bastantes pero no quiere decir que sean anómalos. En conclusión, los clusteres anómalos podrían ser el I, II, III y IV.

Continuando con el dataset **Firewall** y de cara a realizar los modelos, se ha hecho uso del dataset que previamente se había preparado para el modelo de K-means, es decir, el conjunto de datos escalado tras haber eliminado ciertas columnas y con la variable “Action” dumificada. A continuación, se ha entrenado el dataset entero con el modelo Isolation Forest añadiendo el parámetro “contamination = 0.2” para que devolviese un **20% de las instancias totales como anomalías**. Este modelo ha devuelto un total de 11372 observaciones anómalas y 45486 normales.

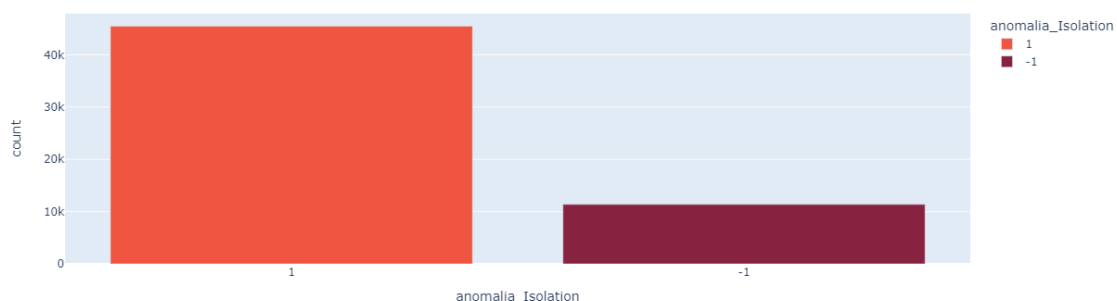


Figura XI - Anomalías detectadas por IsolationForest en Firewall

Además, el modelo IsolationForest asigna **puntuaciones** a las instancias predichas como anómalas en función de cuán anómalas son. En este caso, las puntuaciones obtenidas son las siguientes.

Mediante la siguiente imagen se puede observar que la mayoría de las puntuaciones están por encima de 0 y eso significa que los datos son menos anómalos o lo que es lo mismo. En cambio, los que son negativos muestran que esos valores son más anómalos.

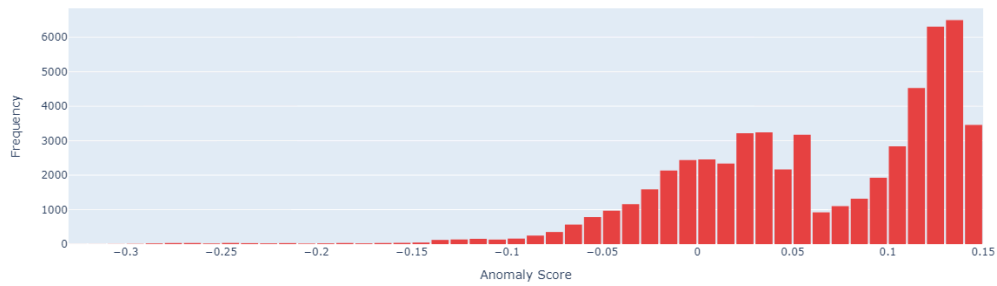


Figura XII - Distribución de las puntuaciones de anomalías por IsolationForest en Firewall

Con el fin de lograr unos resultados más precisos y predecir de forma correcta las anomalías, se ha optado por entrenar otro algoritmo basado en la detección de anomalías, **One Class SVM**. Este modelo ha sido entrenado con las instancias establecidas como no anómalas mediante el algoritmo Isolation Forest, para así poder **predecir sobre las categorizadas como anómalas**. El modelo ha predicho un total de 10452 observaciones anómalas en comparación con las 11372 predichas por Isolation Forest.

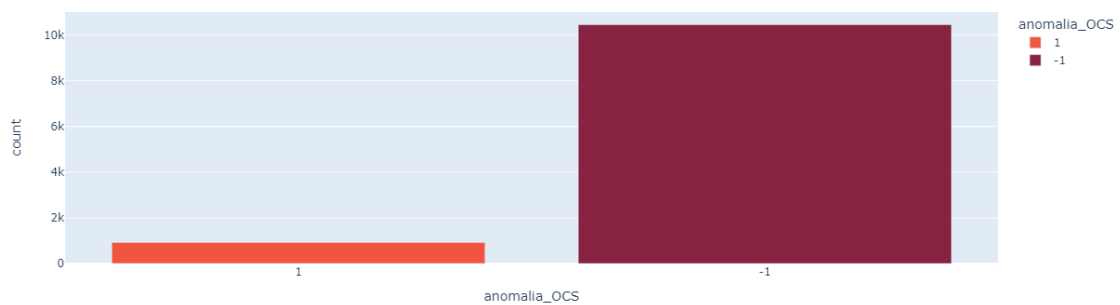


Figura XIII - Anomalías detectadas por OneClassSVM en Firewall

Una vez detectadas las posibles anomalías por los modelos, se han concatenado los datos con una nueva columna denominada “anomalía” que indica -1 si es anomalía y 1 si no lo es. A continuación, se visualizan varias columnas de este dataset dependiendo de la nueva columna de anomalía generada con el fin de saber si estos modelos han hecho un buen trabajo.

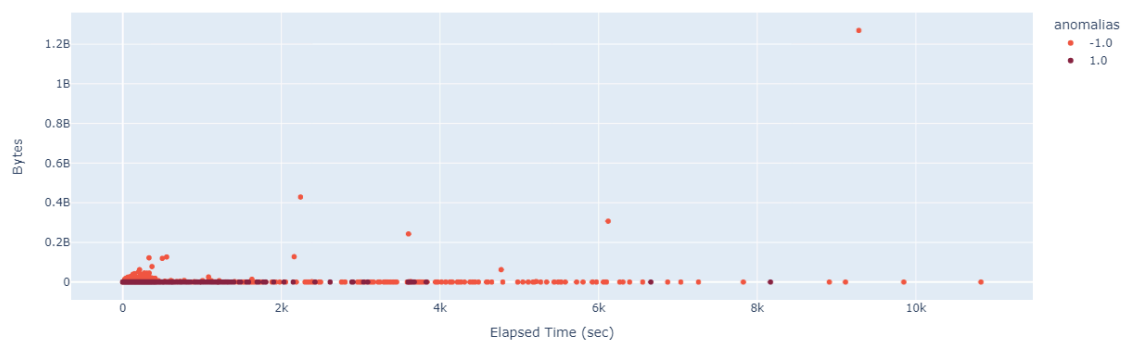


Figura XIV - Gráfico de dispersión Bytes y Elapsed Time por anomalías

Mediante este gráfico de dispersión se puede observar la manera en la que se **distribuyen** las instancias dependiendo el **tipo transcurrido** y los **bytes**. Las anomalías detectadas en este caso se concentran en valores con mucho tiempo transcurrido o con muy poco tiempo. También instancias que tienen valores de bytes muy altos han sido clasificados como anómalos.

4.2.2 Algoritmo Genético

Una vez establecida la estructura se presenta un problema, se deben de garantizar las condiciones laborales de los trabajadores de la planta industrial. Para ello, se dispone de medidores que estiman la frecuencia a la que actúan los compresores A, B, C y D.

Con estos compresores se **regula el volumen de aire necesario** para asegurar condiciones óptimas para los empleados. El objetivo sin embargo es buscar la eficiencia máxima posible, es decir, utilizar el mínimo posible de energía mientras que se mantengan las condiciones laborales establecidas. A fin de lograr dicho objetivo se ha construido un **algoritmo genético junto a la librería PyGad**.

Antes de crear la función fitness se ha generado la **población con datos sintéticos**, estos serán frecuencias con valores aleatorios de entre 0 y 100, siendo una población de 8 individuos y 4 genes. La cantidad de individuos es indiferente, pero no se recomienda que sea muy elevado ya que ralentizaría el resultado de la función, sin embargo, el valor de los genes ha de ser igual al número de compresores, ya que serán las frecuencias de los compresores. Inicialmente, se ha construido una función fitness con las siguientes características:

$$Output = \sum_{i=1}^n (solution_i * volumen de aire_i)$$
$$fitness = - \begin{cases} 10^{10}, & \text{si } output < v \\ \sum predicciones(presión, temperatura, solution), & \text{si } output \geq v \end{cases}$$

Donde v es la consigna de volumen de aire proporcionado y *volumen de aire* una lista con los valores máximos posibles del volumen de aire por cada compresor (100,90,95,110), operando al 100%. El valor de la función fitness debe de ser negativo, ya que el algoritmo genético tiende a maximizar y lo que se busca es minimizar el consumo energético.

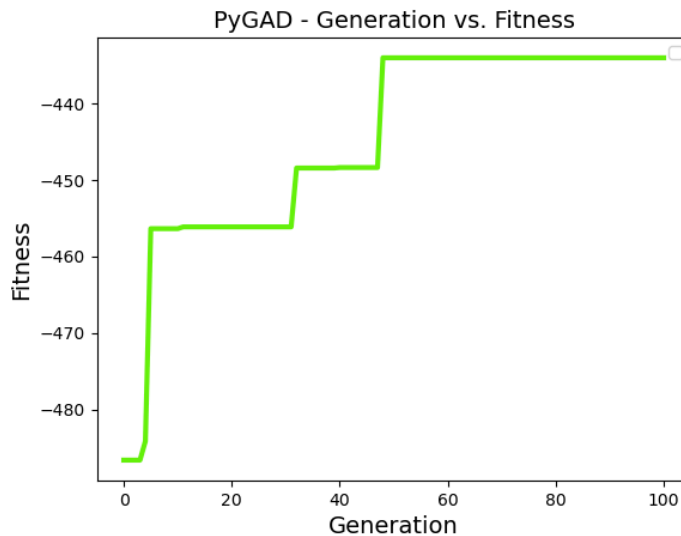


Figura XV - Gráfico de líneas de incremento en la función fitness

Sin embargo, se especifica que los valores de las frecuencias no deben exceder el 90% , con el fin de evitar sobrecargas ya experimentados anteriormente, además de que ningún compresor debe de operar a 0% o 100% ya que no se podrían variar hacia ambos sentidos. Además, se especifica que se sumarán al consumo de energía total 3W por cada compresor que cambie desde su estado inicial, siendo su estado inicial (50, 50, 50, 50), por lo que la función fitness pasará a ser así.

$$\text{fitness} = \begin{cases} 10^{10}, & \text{si } (\text{output} < v) \text{ o (al menos un valor en solution} > 89 \text{ o } < 1) \\ (\sum \text{predicciones(presión, temperatura, solution)}) + 3 \times \text{cambios}, & \text{en otro caso} \end{cases}$$

Dónde *cambios*, es el número de compresores diferentes a la situación inicial, por ejemplo, si tras el algoritmo genético , la solución es (40, 20, 50, 80) , se le sumaría al fitness 9 vatios.

Una vez establecida la función fitness, queda determinar cómo hacer las predicciones, para ello se ha seleccionado el mejor modelo posible para cada compresor a través de un bucle, **comparando los R² de cada modelo con cada compresor**, seleccionando el mayor para cada compresor. En este caso, ningún modelo tarda en exceso como para prescindir de él, por lo que simplemente se elegirá el mejor. [\[Figura IV.1\]](#)

A la hora de realizar las predicciones los valores *Temperatura* y *Presión* serán valores aleatorios dentro del rango permitido por la muestra de datos obtenida, por lo que cada vez que se ejecute el programa , el resultado será diferente. Sabiendo esto, se ha realizado una **búsqueda para los mejores parámetros posibles**, iterando la mayoría de elementos disponibles en los

parámetros y todas sus combinaciones de la función PyGad. Dando como resultado los siguientes argumentos:

Parámetro	Argumento o Valor
Mutation Type	Random
Mutation Percent Genes	10
Mutation Probability	0.3
Crossover Type	two_points
Parent Selection Type	tournament

Tabla IV - Mejores parámetros para el algoritmo genético probando todas las combinaciones posibles

No obstante, esta iteración se ha completado en alrededor de dos horas, y teniendo en cuenta que hay valores aleatorios que influyen directamente en el resultado final, la solución obtenida **es ambigua** y por lo tanto interpretable, ya que cada vez que se realice esta iteración el resultado será diferente, por lo que **es imposible saber cual es la mejor combinación**.

En definitiva, esta búsqueda de parámetros es **computacionalmente costosa** para el beneficio que puede tener, además de que los resultados son ambiguos. Por lo que, en vez de buscar todas las combinaciones posibles se buscará el mejor argumento individualmente para cada parámetro, evitando realizar iteraciones innecesarias.

Para ello, se ha iterado hasta 3 veces cada argumento a fin de tratar de reducir la aleatoriedad en el algoritmo, aún así, los resultados siguen sin ser concluyentes como para asegurar que son los mejores parámetros existentes, no obstante y según las iteraciones realizadas, son los mejores parámetros disponibles. Consiguiendo así, una combinación con los siguientes argumentos:

Parámetro	Argumento o Valor
Mutation Type	Random
Mutation Percent Genes	1
Mutation Probability	0.6
Crossover Type	single_point
Parent Selection Type	tournament

Tabla V - Mejores parámetros para el algoritmo genético probandolos individualmente

Sin embargo, aun con “buenos” parámetros queda la posibilidad de que dada una consigna de volumen de aire muy elevada NO llegue a una combinación de frecuencias que logre asegurar el objetivo. Para solucionar este problema, se puede o bien, ampliar la población inicial para que el algoritmo tenga más posibilidades de encontrar una solución óptima directamente en la

población inicial o bien especificar a través del parámetro *“gene_space”*, siendo una lista de [0,100] , de esta manera el modelo sabrá dónde buscar. Cabe destacar que esta herramienta no asegura encontrar un resultado óptimo siempre y habrá muchas ocasiones en las que tras las 100 generaciones no encontrará resultado, ya que será demasiado específico o simplemente no existirá un resultado para un volumen de aire tan elevado.

4.3 Creación de la infraestructura

El objetivo de la creación de la **infraestructura** es **indexar** datos de varios archivos en **Elasticsearch** utilizando **Logstash** para su posterior análisis y visualización en **Kibana**.

Se han seleccionado todos los Datasets ya que de esta manera, se muestran diferentes aspectos operativos y de seguridad de la red. Los Apache Logs permiten monitorizar el tráfico web, mejorando el rendimiento y la seguridad del sitio. Los SNMP Traps facilitan la gestión de la red, permitiendo arreglar rápidamente los problemas que surjan. Los Firewall Logs proporcionan una visión detallada del tráfico de red, y mediante el uso de **patrones de tráfico**, se detectan amenazas con una mayor facilidad. Con lo cual, se consigue un análisis exhaustivo de la red con el fin de optimizar tanto la seguridad como el rendimiento.

Para la indexación de estos archivos mencionados anteriormente, se ha creado un archivo de configuración de Logstash (logstash.conf).

Este archivo consta de tres secciones:

- En la sección de **input**, se especifican dos bloques file para obtener archivos .txt y .csv del directorio proporcionado.
- En el apartado de **filter**, dependiendo del archivo, se aplican diferentes filtros para estructurar los datos. Para los Apache Logs, se utiliza el filtro grok para desglosar los registros en campos específicos y un filtro date para cambiar de formato a la fecha. Para los SNMP Traps, se utiliza un filtro grok para extraer campos específicos de los registros. Por último, en los Firewall Logs y Network Flows, se utiliza un filtro csv para estructurar los datos en columnas predefinidas.
- En la última sección que corresponde al **output**, se definen dos salidas: una salida estándar (stdout) y otra para indexar los datos en Elasticsearch.

Para obtener detalles adicionales sobre la Incorporación de Información Geográfica a los datos mediante Direcciones IP y configuración del Telebot, se proporcionan en el [Apendice II](#) del presente informe.

4.4. Visualizar los resultados y Conclusiones

4.4.1. Conclusiones anomalías

Con el objetivo de detectar anomalías en los archivos de Apache y Firewall, se han empleado dos herramientas de machine learning para ello. Por un lado, se ha utilizado una combinación del algoritmo de clustering K-means con el Silhouette. Por otro lado, se ha empleado el algoritmo Isolation Forest junto con One-Class SVM.

En cuanto al fichero de Apache, ninguno de los conjuntos de algoritmos ha logrado detectar comportamientos fuera de lo común. Por lo tanto, para abordar este problema, se propone la creación de reglas o agrupamientos lógicos utilizando métricas para prevenir dichos comportamientos anómalos. Un ejemplo de esto mismo podría ser agrupar los registros por la variable temporal y dirección IP del cliente, y luego considerar otras variables como el método o el estado.

En el caso de Apache, el algoritmo K-means es bastante generalista y no consigue formar un clúster con comportamientos anómalos. Por otro lado, la combinación de Isolation Forest con One-Class SVM, en términos generales, detecta anomalías de manera coherente.

4.4.2. Conclusiones algoritmo genético

Durante el reto se ha trabajado dos tipos de bases de datos, por un lado **MySQL**, una **base de datos relacional** que estructura los datos en filas y columnas, este tipo de bases de datos es ideal cuando la información es estructurada y no tiene ramificaciones.

Por otro lado está **MongoDB**, la cual se basa en datos no estructurados mediante documentos JSON, por lo que los elementos dentro de este tipo de base de datos no tienen porque tener la misma longitud y estructura, esto permite una **mayor flexibilidad** si se están guardando datos que pueden modificarse con el tiempo. En este caso, se ha decidido utilizar MongoDB por diferentes razones. En primer lugar, la primera razón por la que se ha utilizado es por la sencillez a la hora de realizar las consultas, ya que como se dispone de una gran cantidad de datos estas consultas pueden ser lentas y dar problemas a la hora de realizar uniones entre tablas. Además, las consultas que se pueden realizar en MongoDB son más complejas ya que tiene más opciones de consultas.

Otra de las ventajas que se han observado de MongoDB respecto a MySQL, es que una de las variables de nuestro df contenía una columna que era una **lista de frecuencias**. MongoDB facilita realizar consultas sobre esta columna ya que guarda esa lista como un diccionario dentro de un diccionario, haciéndolo más accesible y sencillo.

A fin de darle una finalidad real al **algoritmo genético**, se ha creado un servidor **Dash** vinculado a la base de datos, dónde el usuario le inserta los datos de *presión*, *temperatura* y *volumen de aire objetivo*, dando como resultado las **frecuencias de cada compresor**, este resultado, será reflejado a través de un gráfico de barras y uno de radar mientras que la información del algoritmo se guarda en la base de datos, de manera que la próxima vez que se ingresen datos ya registrados en la base de datos, **no será necesario ejecutar el algoritmo genético**, y se devolverán los gráficos pertinentes, facilitando y agilizando la tarea. En conclusión, es gracias a Dash y a MongoDB que el algoritmo genético tiene una aplicabilidad real, y de posible uso directo del usuario.

4.4.3. Conclusiones Infraestructura

Mediante la previa detección de anomalías, en la que se pueden detectar patrones diferentes a los normales y la infraestructura Big Data, **es posible identificar un potencial ataque a la ciberseguridad del usuario de manera proactiva**. La infraestructura Big Data permite manejar y analizar grandes volúmenes de datos en tiempo real, lo que facilita la identificación rápida y precisa de actividades sospechosas. Estas anomalías pueden incluir comportamientos inusuales en el tráfico de red, accesos no autorizados, u otros indicios de posibles amenazas.

4.5. Sistema publisher/subscriber

Para complementar la **infraestructura de Big Data** que se ha explicado anteriormente, se ha decidido crear un sistema publisher/subscriber utilizando Mosquitto, un broker de mensajes de código abierto que implementa el protocolo **MQTT**. El objetivo de este sistema es realizar consultas sobre los datos de los compresores, para luego indexar los resultados de estas mismas en Elasticsearch, con el fin de visualizarlo mediante un **dashboard de Kibana**.

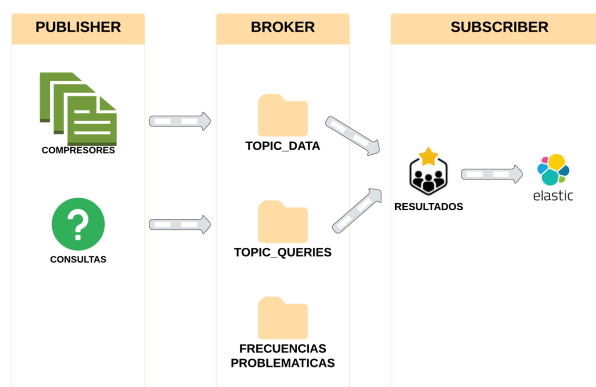


Figura XVI - Diagrama del sistema publisher/subscriber

En la Figura XII, se puede observar cómo está estructurado este sistema publisher/subscriber. En primer lugar, se puede observar como se ha creado un **broker** donde se han planteado tres topics diferentes: “topic_data”, “topic_queries” y “frecuencias_problematicas”. Estos topics

serán los encargados de gestionar la comunicación entre el publisher y el subscriber, y el contenido de cada uno se explicará a continuación.

Por un lado, se puede apreciar que existe un **client_1**, el cual tiene como función **publicar los datos de los compresores**, los cuales, al disponer de cuatro datasets diferentes, uno por cada compresor, se han juntado previamente en un mismo dataframe. Sin embargo, al tener una gran cantidad de datos por compresor, lo que provocaba problemas en la máquina virtual y un tiempo de espera considerable hasta que se publicasen todos los datos, se ha decidido publicar solamente 200 filas por cada compresor, ya que esta se ha considerado una muestra lo suficientemente representativa como para reflejar la actitud de cada compresor. El **client_1** va publicando los datos en el topic llamado “topic_data” con una **frecuencia de 1 segundo**. Asimismo, se le ha añadido una variable de tiempo a los datos con el fin de observar la fecha en la que se ha publicado cada fila.

Por otro lado, el **client_2** es el encargado de **realizar las consultas**, que los resultados de estas, posteriormente, se **indexarán en elasticsearch**, como bien se ha mencionado anteriormente. Estas consultas se publican en el “topic_queries”. El contenido de estas consultas se explicará en el [Apéndice III](#).

Por último, el **subscriber** es el encargado de estar **suscrito a los topics generados para reflejar el contenido de las consultas**. Además, es el encargado de guardar todos los datos en el que los compresores actúan a una frecuencia de 0 o entre 90 y 100 en el topic llamado “frecuencias problemáticas”. Se ha decidido crear este topic, ya que, como bien se ha mencionado en la explicación del algoritmo genético, es recomendable evitar que los compresores actúen a estas frecuencias con el fin de **evitar sobrecargas**.

6. Implicaciones legales y éticas

Incorporar aspectos éticos en el manejo de datos es esencial para Lanit, no solo desde una perspectiva de **cumplimiento normativo**, sino también como un **valor añadido**. Esto puede fortalecer la **confianza** de clientes y socios, diferenciando a Lanit en el mercado. Promover la ética en el trabajo con datos puede generar una **imagen positiva y responsable** de la empresa, alineándose con los valores de **transparencia** y **respeto a la privacidad**.

En el contexto de la automatización en plantas industriales, es evidente que un mayor nivel de automatización implica una **reducción de la mano de obra humana**. Este nuevo modelo industrial se basa en la mayor eficiencia y una reducción significativa de los costos de operación, aunque también presenta **la desventaja de la pérdida de empleos** y la necesidad de reentrenamiento de la fuerza laboral. Como expertos en tecnologías y soluciones informáticas, Lanit puede desempeñar un papel vital en la transición hacia la automatización y digitalización, proporcionando herramientas y métodos que promuevan una **mejor integración de la fuerza laboral** en las plantas industriales automatizadas.

En cualquier modelo de monitoreo y conectividad, la **ciberseguridad** es un elemento clave para Lanit. La **protección de datos y sistemas** es crítica para evitar que los sistemas caigan en manos de piratas informáticos. Además, al implementar sensores y realizar un tratamiento masivo de datos, se incrementa el consumo energético. Por tanto, Lanit debe no solo fortalecer sus estrategias de ciberseguridad para proteger la monitorización y los sistemas conectados contra amenazas cibernéticas, sino también **evaluar** cuidadosamente si el **consumo de energía** para el procesamiento de datos supera las mejoras obtenidas en la eficiencia energética de las infraestructuras. Esto implica la búsqueda de soluciones que **minimicen el consumo energético** en el análisis de datos, **optimizando así el balance entre consumo y ahorro energético**.

Lanit debe ser **transparente** en sus decisiones, proporcionando informes claros y accesibles para generar confianza. Es esencial advertir a los clientes sobre las limitaciones del análisis de datos y establecer **expectativas realistas**. Para demostrar su compromiso con la ética y la sostenibilidad, Lanit debería integrar una sección dedicada a estos temas y a los **Objetivos de Desarrollo Sostenible** (ODS) en su página web. En cuanto al impacto en los ODS, Lanit puede mejorar la **eficiencia y sostenibilidad** de las infraestructuras industriales (ODS 9), aunque debe abordar la posible reducción de empleos (ODS 8) mediante la **creación de empleos tecnológicos**. Además, puede optimizar el uso de recursos a través del análisis de datos, **fomentando la producción y el consumo responsables** (ODS 12).

La creación de un **comité ético** para supervisar el análisis de datos es altamente recomendable para **asegurar prácticas responsables** y alineadas con los valores éticos y los ODS.

Bibliografía

Arpit. (2021, 13 diciembre). Isolation Forest algorithm for anomaly detection - Arpit - Medium. <https://medium.com/@arpitbhayani/isolation-forest-algorithm-for-anomaly-detection-f88af2d5518d>

Bismi, I. (2023, September 16). Detecting Anomalies: A Comprehensive Guide with One-Class SVM and Isolation Forest Medium. <https://ai.plainenglish.io/detecting-anomalies-a-comprehensive-guide-with-one-class-svm-and-isolation-forest-230336f0988a>

Dey, R. (2024, 17 marzo). Anomaly Detection using Support Vectors - Roshmita Dey - Medium. <https://medium.com/@roshmitadey/anomaly-detection-using-support-vectors-2c1b842213ed>

Durvasula, V. (2023, 27 diciembre). Understanding One-Class SVM for Anomaly Detection - AskPython. <https://www.askpython.com/python/examples/one-class-svm-anomaly-detection>

Elastic. (2022). ELK Stack: Elasticsearch, Kibana, Beats y Logstash. <https://www.elastic.co/es/elastic-stack>

Grupo Lanit. (2022, 18 septiembre). Lanit - consultoría tecnológica y de negocio. <https://www.lanit.eu/sobre-lanit/>

Heuristic algorithms - Cornell University Computational Optimization Open Textbook - Optimization Wiki. (s. f.). https://optimization.cbe.cornell.edu/index.php?title=Heuristic_algorithms

IsolationForest. (s. f.). Scikit-learn. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html>

Jecrespom. (2021, 27 febrero). Mosquitto. Aprendiendo Arduino. <https://aprendiendoarduino.wordpress.com/2018/11/19/mosquitto/#:~:text=Mosquitto%20es%20un%20broker%20MQTT,%C3%A9stos%20son%20de%20pocos%20recursos.&text=A%20trav%C3%A9s%20de%20esta%20p%C3%A1gina,cual%20sea%20tu%20sistema%20operativo.>

Kagumire, S. (2022, 6 enero). Anomaly Detection with Machine Learning - Sulaiman Kagumire - Medium. <https://sulaimank.medium.com/anomaly-detection-with-machine-learning-8fa942fb5adc>

Oracle. (2024). ¿Qué es el big data? <https://www.oracle.com/es/big-data/what-is-big-data/>

Ramírez, I. (2020, 31 enero). Máquinas virtuales: qué son, cómo funcionan y cómo utilizarlas. Xataka. <https://www.xataka.com/especiales/maquinas-virtuales-que-son-como-funcionan-y-como-utilizarlas>

Rodríguez, D. (2024, 7 abril). One-Class SVM: Detección de anomalías con máquinas de vector soporte. Analytics Lane. <https://www.analyticslane.com/2024/03/15/one-class-svm-deteccion-de-anomalias-con-maquinas-de-vector-soporte/>

Sebastián, C. S. J. (2023). Detección de anomalías transaccionales usando técnicas de machine learning con grafos. repository.urosario.edu.co. https://doi.org/10.48713/10336_40983

V, B., & V, B. (2023, 26 septiembre). ¿Qué es Ubuntu? Una guía rápida para principiantes. Tutoriales Hostinger. <https://www.hostinger.es/tutoriales/que-es-ubuntu>

V, M. (2024, 2 abril). A comprehensive guide for SVM One-Class Classifier for anomaly Detection. AnalyticsVidhya. <https://www.analyticsvidhya.com/blog/2024/03/one-class-svm-for-anomaly-detection/>

What Is One Class SVM and How Does It Work? (s. f.). Baeldung. <https://www.baeldung.com/cs/one-class-svm>

Apéndice I

Con el objetivo de dar respuesta a los objetivos planteados en el Reto 8 y haciendo uso de **modelos matemáticos**, se ha procedido a modelar los datos del dataset Firewall de cara a encontrar, fundamentar y tratar las anomalías de los datos.

En este dataset de tráfico de red, podrían existir **valores anómalos** en cuanto a puertos de origen o destino no usuales en el tráfico. Por otra parte, también podría resultar sospechoso si el número de bytes enviados o recibidos es significativamente menor o mayor en comparación con otros registros similares. Además, el **desbalance** significativo entre la cantidad de paquetes enviados y recibidos también puede llegar a resultar una anomalía en el conjunto de datos. Asimismo, la presencia de acciones "deny" en lugar de "allow" en registros que normalmente deberían estar permitidos, así como inconsistencias en los puertos NAT, pueden indicar intentos de acceso no autorizado, configuraciones erróneas del firewall, o actividad sospechosa en la red.

Teniendo esto en cuenta y conociendo la relevancia de la información aportada por el conjunto de datos, se han empleado dos **modelos de detección de anomalías**: Isolation Forest y One-Class SVM.

Isolation Forest

El modelo de detección de anomalías Isolation Forest, basado en el principio de que las anomalías son pocas y distintas, es una buena técnica para **identificar comportamientos inusuales** en conjuntos de datos grandes y variados, ya que su **enfoque de aislamiento** permite detectar anomalías de manera eficiente sin requerir supuestos sobre la distribución de los datos.

Este modelo se basa en la **construcción de árboles de aislamiento** (iTrees). La idea fundamental es que las anomalías son más fáciles de aislar que las observaciones normales debido a sus **características distintivas**. Para aislar un punto, se selecciona una característica al azar y se divide aleatoriamente su rango de valores. Este proceso se repite hasta que todos los puntos quedan aislados en nodos individuales.

Construcción de iTrees

1. Selección de características y partición:

Siendo $X = \{x_1, x_2, \dots, x_n\}$ el conjunto de datos con n observaciones

Para construir un árbol, se selecciona **aleatoriamente** una característica f de los d posibles, y se elige un valor de partición p aleatoriamente dentro del rango de valores de esa característica.

Se crean dos particiones del conjunto de datos: X_L y X_R , donde:

$$X_L = \{x \in X : x_f < p\}$$

$$X_R = \{x \in X : x_f \geq p\}$$

2. Repetición del proceso:

El proceso de selección de características y partición **se repite recursivamente** hasta que todas las observaciones estén aisladas en **nodos individuales** o se alcance una **profundidad máxima predefinida** (H_{max}).

Profundidad del árbol y anomalías

La clave del Isolation Forest es la **relación entre la profundidad del árbol** (el número de divisiones necesarias para aislar una observación) **y la probabilidad de que una observación sea una anomalía**. Las anomalías tienden a aislarse más cerca de la raíz del árbol porque requieren menos particiones para quedar solas. Esta relación se describe mediante la función armónica $H(n)$, que estima la profundidad esperada $E(h(x))$ de una observación x en un Isolation Tree

$$E(h(x)) \approx 2 \cdot H(n - 1) - \left(\frac{2(n-1)}{n} \right)$$

donde $H(i)$ es la i -ésima constante armónica:

$$H(i) = \sum_{k=1}^i \frac{1}{k}$$

Puntuación de anomalías

Para determinar si una observación es una anomalía, se utiliza la puntuación de anomalía $s(x, n)$ definida como:

$$s(x, n) = 2^{-\frac{E(h(x))}{c(n)}}$$

donde $c(n)$ es la profundidad media de un Isolation Tree dado n :

$$c(n) = 2 \cdot H(n - 1) - \left(\frac{2(n-1)}{n} \right)$$

Una puntuación cercana a 1 indica que la observación es probablemente una anomalía, mientras que una puntuación cercana a 0 indica que es una observación normal.

Funcionamiento general del algoritmo

En primer lugar se construye el bloque. Se generan t árboles de aislamiento (iTrees), cada uno construido mediante la selección aleatoria de una muestra del conjunto de datos y la aplicación del proceso de partición previamente descrito.

En la evaluación de anomalías, se calcula la **profundidad promedio** $E(h(x))$ para cada observación x a través de los t árboles. Esta profundidad promedio se utiliza para calcular la **puntuación de anomalía** $s(x, n)$. El Isolation Forest es eficiente y escalable, con una complejidad temporal de $O(t \cdot \psi \cdot \log(\psi))$, donde ψ es el tamaño de la muestra utilizada para construir cada árbol, y t es el número de árboles en el bosque.

One Class SVM

El modelo de One Class SVM (Support Vector Machine) es una **técnica de aprendizaje automático** utilizada para la **detección de anomalías en datos no etiquetados** al encontrar un **hiperplano** que delimite la región que contiene la mayoría de los datos normales. La SVM busca **maximizar la distancia entre este hiperplano y los puntos más cercanos a él**, conocidos como vectores de soporte. Los puntos que quedan fuera de esta región, en el lado opuesto del hiperplano, se consideran **anomalías**.

Formulación matemática

Dado un conjunto de datos $X = \{x_1, x_2, \dots, x_n\}$ en un espacio de características $\mathbb{R}^d$, siendo d la dimensión de los datos, la One Class SVM encuentra el hiperplano óptimo mediante la minimización de la función de costo regularizada:

$$\min(w, p) \frac{1}{2} \|w\|^2 + \frac{1}{vn} \sum_{i=1}^n \xi_i - p$$

sujeto a las **restricciones**:

$$w^T \phi(x_i) \geq p - \xi_i, \quad \xi_i \geq 0, \quad \sum_{i=1}^n \xi_i \leq vn$$

donde:

- w es el **vector normal al hiperplano**.
- p es la **distancia del hiperplano al origen**.
- $\phi(x)$ es la **función de mapeo** que proyecta los datos a un espacio de características de mayor dimensión (puede ser lineal o no lineal).
- ξ_i son **variables de holgura** que permiten que algunos puntos estén dentro de la región delimitada por el hiperplano.

- ν es un **parámetro de regularización** que controla el porcentaje esperado de datos atípicos.

La resolución de la One Class SVM se realiza mediante técnicas de **optimización convexa** y la formulación del Lagrangiano dual, lo que permite hallar los coeficientes α^i que caracterizan los vectores de soporte.

Una vez obtenidos estos **coeficientes**, se calcula el hiperplano de separación y se determina si una nueva observación x es una anomalía mediante el **cálculo de su distancia al hiperplano**.

Entre las ventajas de la One Class SVM, se destaca su capacidad para manejar **conjuntos de datos de alta dimensión** y encontrar regiones de alta densidad en el espacio de características. No obstante, presenta limitaciones, como su sensibilidad a la elección de parámetros, como el parámetro de regularización ν , y su posible costo computacional en grandes conjuntos de datos.

En resumen, One Class SVM es una **herramienta eficaz para la detección de anomalías** al identificar la región que engloba la mayoría de los datos normales en un espacio de características.

Kmeans

El algoritmo K-Means es una **técnica de agrupamiento empleada en aprendizaje no supervisado** para dividir un conjunto de datos en k grupos distintos. Inicia seleccionando k **centroides aleatorios**, los cuales representan los centros iniciales de los grupos en el espacio de características. Estos centroides son **vectores** en el mismo espacio dimensional que los datos, y su elección puede influir en la **convergencia y calidad de los grupos** resultantes.

Asignación de puntos a grupos

En la etapa de asignación de puntos a grupos, cada punto de datos x_i se asigna al grupo cuyo **centroide está más cercano**. Esto se realiza calculando la distancia entre cada punto y cada centroide, utilizando típicamente la distancia euclidiana. Formalmente, para cada punto x_i , se calcula la distancia a todos los centroides y se asigna el punto al grupo cuyo centroide tiene la **distancia mínima**. Matemáticamente, la asignación de un punto x_i al grupo j se expresa como:

$$c_i = \arg \min_j \|x_i - \mu_j\|^2$$

Donde c_i es el índice del grupo al que se asigna x_i , μ_j es el centroide del grupo j , y $\|\cdot\|$ representa la **distancia euclidiana**.

Actualización de Centroides

Después de asignar todos los puntos a los grupos, se **actualizan los centroides de cada grupo** para reflejar la ubicación media de los puntos asignados a ese grupo. Este proceso de actualización de centroides se lleva a cabo calculando la media de todos los puntos en cada grupo y ajustando la posición del centroide en consecuencia. La **nueva posición del centroide** se calcula mediante:

$$\mu_j = \frac{1}{|S_j|} \sum_{x_i \in S_j} x_i$$

Donde:

- μ_j es el nuevo centroide del grupo j .
- S_j es el conjunto de puntos asignados al grupo j .
- $|S_j|$ es el tamaño del conjunto de puntos asignados al grupo j .

Proceso iterativo

El algoritmo itera entre las etapas de asignación de puntos a grupos y actualización de centroides hasta que se cumple un criterio de detención. Este criterio puede ser un número **máximo de iteraciones o hasta que los centroides convergen**, es decir, cuando ya no cambian significativamente entre iteraciones. La convergencia garantiza que el algoritmo haya encontrado una solución estable.

El objetivo final del algoritmo K-means es **minimizar la suma de las distancias al cuadrado entre cada punto y el centroide de su grupo asignado**. Esta medida, conocida como inercia o suma de cuadrados dentro del cluster (SSW), cuantifica la **cohesión** de los grupos. Minimizar la inercia implica formar **grupos compactos y bien separados**, donde los puntos dentro de cada grupo son similares entre sí y diferentes de los puntos en otros grupos. La función objetivo que el algoritmo intenta minimizar es:

$$J = \sum_{j=1}^k \sum_{x_i \in S_j} \|x_i - \mu_j\|^2$$

Esta función J es la suma total de las varianzas dentro de cada grupo. Al minimizar esta función, el algoritmo K-means encuentra los **grupos óptimos**.

Algoritmo genético

Para dar respuesta al objetivo planteado de **optimizar el uso de energía de los compresores A, B, C y D** en una planta industrial para asegurar las condiciones laborales de los empleados, se ha empleado un **algoritmo genético** implementado con la librería **PyGad**. El objetivo es minimizar el consumo de energía mientras se garantiza un volumen de aire adecuado, evitando sobrecargas y asegurando la variabilidad en la operación de los compresores.

Para comenzar, se genera una **población inicial con datos sintéticos**. Cada individuo en la población representa una posible solución y está compuesto por 4 genes que corresponden a las frecuencias de los compresores. Los valores de estos genes son **aleatorios** y oscilan entre 0 y 100, con una población inicial de **8 individuos**. Es importante que la cantidad de individuos no sea excesivamente alta para evitar que la función se vuelva demasiado lenta, aunque debe ser suficiente para proporcionar una variedad adecuada de soluciones.

La **función fitness** es clave en este proceso y se define para evaluar cada solución en términos de consumo energético. La fórmula básica de la función fitness es la **suma ponderada de las frecuencias de los compresores por los volúmenes de aire que proporcionan al 100% de su capacidad**. Esto se expresa matemáticamente como:

$$fitness = - \left(\sum_{i=1}^4 (f_i \cdot V_i) \right)$$

Donde f_i es la frecuencia del compresor i y V_i es el volumen de aire proporcionado por cada compresor operando al 100%. Los valores máximos de los volúmenes de aire para los compresores son 100, 90, 95 y 110 respectivamente. La función fitness se multiplica por -1 porque el algoritmo genético de PyGad tiende a maximizar la función fitness y lo que se busca aquí es **minimizar el consumo energético**.

Además, se imponen restricciones para asegurar que los valores de las frecuencias no excedan el 90% ni caigan por debajo del 10%, para evitar sobrecargas y asegurar la variabilidad en la operación. Adicionalmente, se introduce una penalización de 3W por cada compresor cuya frecuencia difiera de su estado inicial (50, 50, 50, 50). Esta penalización se añade al consumo de energía total para reflejar los cambios en el estado de los compresores:

$$fitness = - \left(\sum_{i=1}^4 (f_i \cdot V_i) + 3 \times cambios \right)$$

Donde *cambios* representa el número de compresores con frecuencias diferentes al 50%.

El **proceso del algoritmo genético** se compone de varias etapas:

1. **Inicialización:** generación de una población inicial con frecuencias aleatorias entre 10 y 90 para evitar los extremos 0% y 100%.
2. **Evaluación:** cálculo de la función fitness para cada individuo en la población.
3. **Selección:** selección de los individuos para el cruce, utilizando el método de torneo.
4. **Cruzamiento:** intercambio de genes entre pares de individuos para generar nueva descendencia, utilizando un cruce de un solo punto (single-point crossover).
5. **Mutación:** modificación aleatoria de algunos genes para mantener la diversidad genética, con una probabilidad de mutación del 60% y afectando al 1% de los genes.
6. **Iteración:** repetición de este proceso hasta alcanzar el número máximo de generaciones (100).

Para encontrar los mejores parámetros del algoritmo genético, se ha probado una **variedad de combinaciones**, ajustando individualmente cada parámetro para **reducir la aleatoriedad**. Los parámetros finales seleccionados han sido: tipo de **mutación aleatoria**, un **1% de genes mutados**, una **probabilidad de mutación del 60%**, **cruce de un solo punto**, y selección de **padres por torneo**. Aunque esta metodología no garantiza siempre encontrar una solución óptima, proporciona una **estrategia estructurada** para minimizar el consumo energético de los compresores.

En resumen, el uso de un algoritmo genético con la librería PyGad permite **optimizar el consumo energético** de los compresores, asegurando condiciones laborales óptimas y cumpliendo con restricciones operacionales. Aunque la búsqueda de **parámetros óptimos** es computacionalmente costosa y los resultados pueden ser ambiguos, este enfoque proporciona una herramienta eficiente para **gestionar la energía en la planta industrial**.

Apéndice II

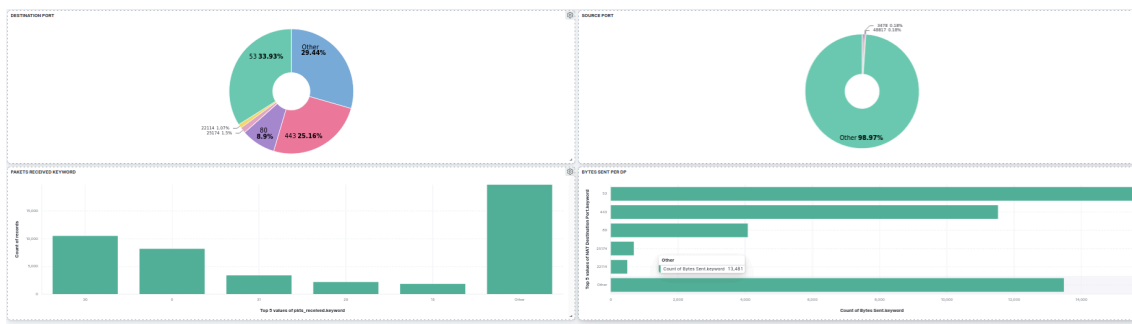
Para obtener el campo del **código regional** que permitirá la visualización en mapas, se ha realizado un proceso.

Primero de todo, se ha realizado una **importación** de datos de registros de Apache y Network Flows utilizando la función “pd.read_csv”. Posteriormente, se han **limpiado** las direcciones IP en los registros de Apache para eliminar cualquier carácter no deseado utilizando la función lambda aplicada a la columna “client_ip”. Se ha creado una lista de direcciones IP únicas extrayendo todas las direcciones IP de las columnas relevantes en todos los datos de Apache y los Network Flows utilizando la función “unique_ips”.

En el cuarto paso, se ha hecho uso de la biblioteca “IP2Location” para obtener **información geográfica**, incluyendo el código regional para cada dirección IP. Esto se ha logrado gracias a la creación de una instancia de la clase “IP2Location.IP2Location” con el archivo de base de datos de IP2Location y una instancia de la clase “IP2Location.Region” con el archivo de **diccionario de regiones**. Una vez habiendo realizado este paso, se han asignado códigos regionales a las direcciones IP, iterando sobre las direcciones IP y mediante la función “get_country”, se ha obtenido el código regional correspondiente a cada dirección IP. En consecuencia, se ha creado un data frame que contiene direcciones IP y sus respectivos códigos regionales.

Antes de finalizar, se ha **unido la información geográfica con los datos originales de Apache y con los de Network Flows** utilizando la función “get_countries”. Por último, los datos finales tras la unión realizada anteriormente, han sido exportados a nuevos archivos CSV para usarlos en la **visualización de mapas**. En el caso de Apache, todos los registros de la columna “client_ip_country” son None. En cambio, en Network Flows se han obtenido múltiples registros en la columna “Source.IP_country” y “Destination.IP_country”, aun habiendo muchos registros None también.

Como último apartado, se ha creado un **bot** para **interactuar con datos de Elasticsearch y generar gráficos**. En primer lugar, se han establecido las configuraciones necesarias para **conectar el bot a Telegram y a Elasticsearch**. Se han definido las variables de configuración, como el token de Telegram y las credenciales de Elasticsearch.



Se ha realizado un Dashboard sobre la base de datos snmp. La visualización se ha realizado sobre el Destination Port, mostrando lo que abarca cada de la variable cada registro diferente.

Puerto de Destino (Destination Port): La preponderancia de puertos específicos (53, 443, 80) sugiere que el tráfico de red está dominado por servicios de DNS, HTTPS y HTTP.

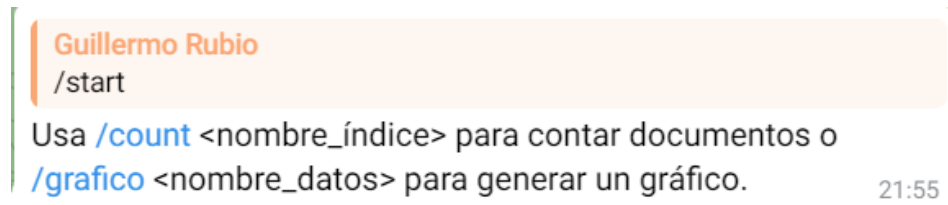
Puerto de Origen (Source Port): La alta concentración en 'otros' indica una gran variedad de puertos de origen, lo que puede reflejar una diversidad en los dispositivos y aplicaciones que generan el tráfico.

Paquetes Recibidos: La distribución de paquetes recibidos con picos en ciertos valores específicos (30, 31) puede indicar patrones de comportamiento o configuraciones comunes en la red.

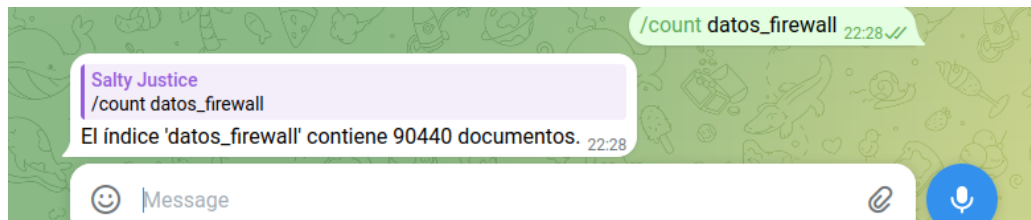
Bytes Enviados: La cantidad de bytes enviados a puertos específicos refuerza la importancia de ciertos servicios (DNS, HTTPS) en la red analizada.

El script incluye cuatro funciones que manejan comandos recibidos por el bot:

La primera función es la de “send_welcome” que responde a los comandos “/start” y “help”, proporcionando instrucciones sobre cómo utilizar el bot.



Como segunda función se ha definido “count_documents”, que maneja el comando “/count” para contar el número de documentos guardados en un índice de Elasticsearch.



Por último, como respuesta al comando “/grafico” se han creado tres diferentes funciones que se van a llamar una detrás de otra. Los únicos data frames sobre los que se harán los gráficos son el de Apache, Firewall y Network. La primera preguntará qué columna se quiere graficar en el eje X, lo mismo con la segunda pero esta vez con el eje Y.

Para la selección de las columnas, se han **filtrado solamente las columnas que son numéricas**, además, se ha creado un ReplyKeyboardMarkup() con el objetivo de hacer más sencilla la selección. Por último, la última creará un gráfico de dispersión con las dos variables seleccionadas. En la imagen de abajo se puede observar el resultado de un ejemplo.

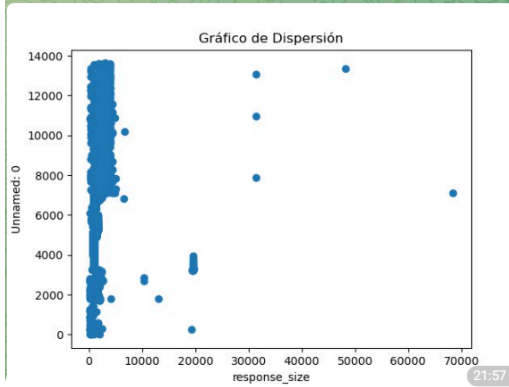
Escoge la variable que quieras graficar en el eje x 21:57

response_size 21:57 ✓

Escoge la variable que quieras graficar en el eje Y 21:57

Unnamed: 0 21:57 ✓

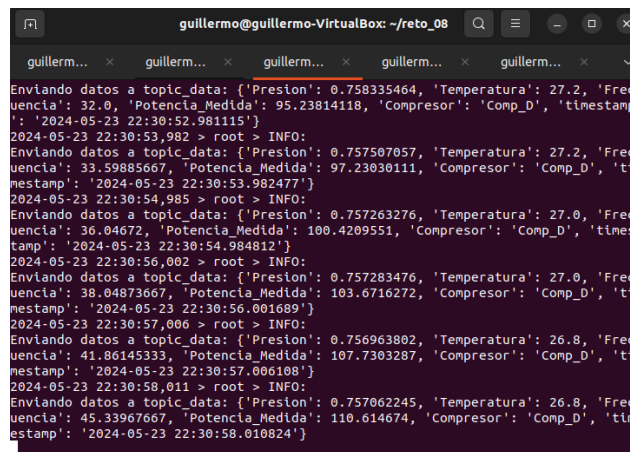
Aquí esta el grafico: 21:57



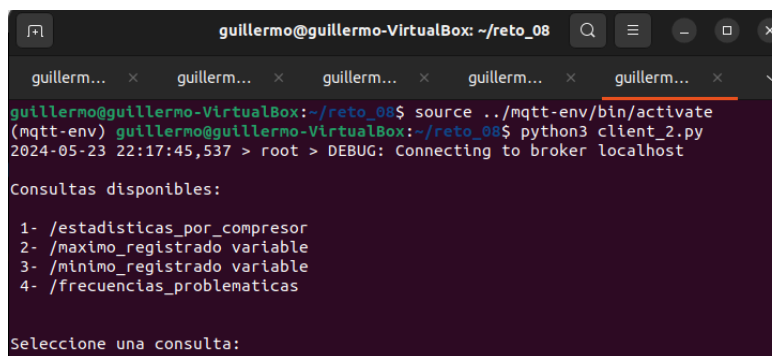
Apéndice III

En este apéndice se explicará la manera de comportarse tanto de los **clientes** como del **subscriber**, es decir, como **publica los datos el client_1**, las **consultas que se pueden realizar** y los **resultados de cada una**. Además, se ha realizado un **dashboard en Kibana** con los resultados de las consultas. Cabe destacar que el orden de ejecución de cada script para que el sistema funcione correctamente es el siguiente: **client_1.py**, **queries.py** y, por último, **client_2.py**.

A continuación, en la siguiente imagen se puede observar como publica los datos el **client_1**.

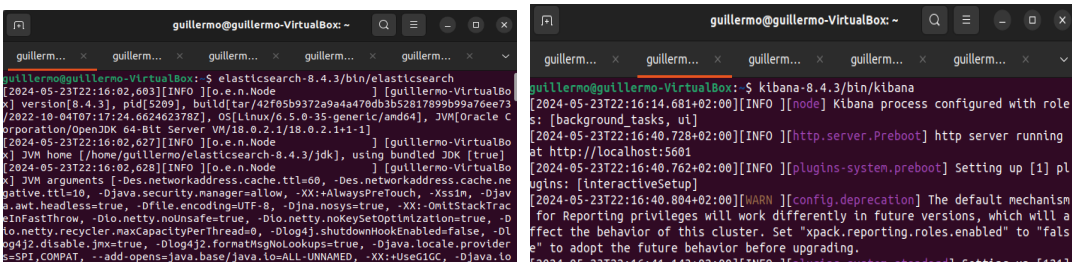
A terminal window titled 'guillermo@guillermo-VirtualBox: ~/reto_08' showing a series of log messages. The messages indicate that data is being sent to a topic named 'data'. Each message is a JSON object containing sensor data: 'Presion', 'Temperatura', 'Frecuencia', 'Potencia_Medida', 'Compresor', and 'timestamp'. The 'Compresor' field is consistently 'Comp_D'. The timestamps are sequential, starting from '2024-05-23 22:30:52.981115'.

Por otro lado, las consultas que se van a realizar son las siguientes:

A terminal window titled 'guillermo@guillermo-VirtualBox: ~/reto_08' showing the execution of a script. The prompt is 'guillermo@guillermo-VirtualBox:~/reto_08\$ source ../mqtt-env/bin/activate (mqtt-env) guillermo@guillermo-VirtualBox:~/reto_08\$ python3 client_2.py'. The output shows the connection to the broker and a list of available queries: 1- /estadisticas_por_compresor, 2- /maximo_registrado variable, 3- /minimo_registrado variable, 4- /frecuencias_problematicas. The prompt 'Seleccione una consulta:' is shown at the bottom.

A continuación, se van a reflejar los resultados de cada consulta. El resultado de la primera consulta, **“/estadisticas_por_compresor”**, es un data frame donde se agrupan los datos por compresor y se calculan la media de presión, temperatura, frecuencia y potencia medida, y el número de datos registrados por cada compresor. Se ha decidido realizar esta consulta ya que podría ayudar a realizar un **breve análisis sobre los compresores**.

Cabe destacar que para que las consultas funcionen es necesario tener tanto Elasticsearch como Kibana **iniciados** de esta manera, ya que los **resultados de las consultas se indexarán en Elasticsearch**.



El resultado de la **primera consulta** quedaría así:

```
Estas son las estadísticas por compresor solamente teniendo en cuenta esas frecuencias:
```

	Compresor	Media_presion	...	Media_potencia	numero_de_datos
0	Comp_A	0.760656	...	81.939474	44
1	Comp_B	0.755490	...	65.675067	61
2	Comp_C	0.749094	...	90.653910	51
3	Comp_D	0.758550	...	80.973770	53

[4 rows x 6 columns]

La segunda y la tercera consulta, **“/maximo_registrado”** variable y **“/minimo_registrado”** variable, requiere que se le indique de qué variable se quiere consultar el máximo/mínimo registrado. Estas consultas pueden ayudar a la hora de **realizar el algoritmo genético**, en cuanto a escoger una presión y una temperatura para las predicciones.

Si se le indica mal la variable el resultado será el siguiente:

```
La variable indicada no es una variable del df. Las variables son las siguientes: Presion, Temperatura, Potencia_Medida
```

Si la variable introducida es correcta los resultados serán de la siguiente manera:

```
El maximo registrado de la variable Presion es 0.743642944 y se registro en el Comp_A
```

```
El minimo registrado de la variable Temperatura es 16.4 y se registro en el Comp_A
```

Por último, la última consulta devuelve el mismo resultado que la primera, sin embargo, en esta última solamente se tienen en cuenta los datos de los compresores que actúan a una **frecuencia de 0 o entre 90 y 100**.

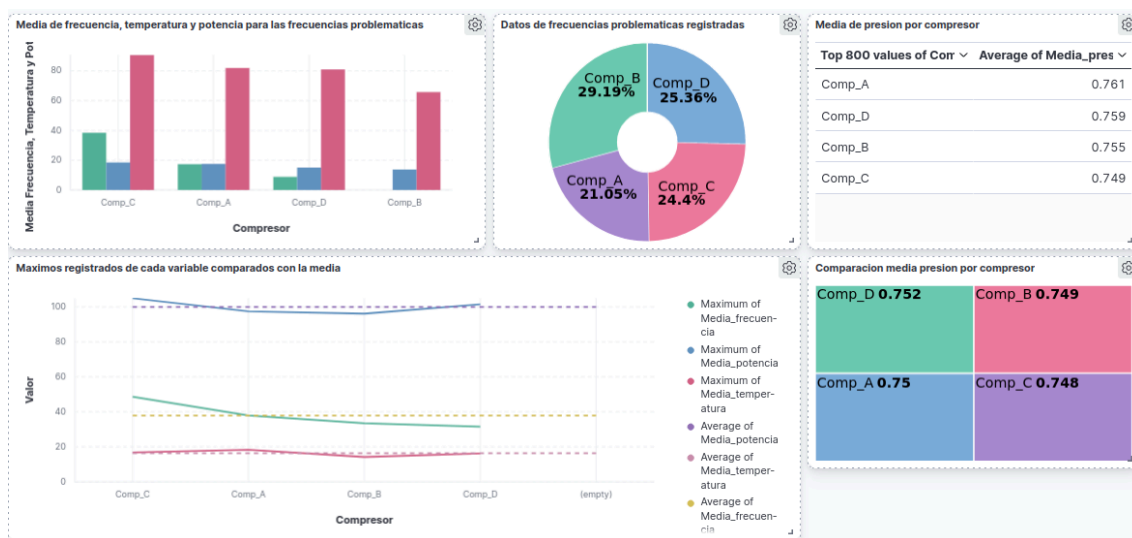
La razón de porqué se ha realizado esta consulta es simplemente para observar qué **compresor es más probable que sufra una sobrecarga debido a una mala frecuencia**. El resultado quedaría así:

```

Estas son las estadísticas por compresor solamente teniendo en cuenta esas frecuencias:
Compresor  Media_presion  ...  Media_potencia  numero_de_datos
0  Comp_A      0.760656  ...      81.939474      44
1  Comp_B      0.755490  ...      65.675067      61
2  Comp_C      0.749094  ...      90.653910      51
3  Comp_D      0.758550  ...      80.973770      53
[4 rows x 6 columns]

```

Con el objetivo de visualizar los resultados obtenidos de las consultas, se ha creado un **dashboard** en Kibana.



Este dashboard visualiza un análisis general de los resultados de las consultas dividido en dos partes. La parte de arriba está dedicada a los gráficos realizados solamente teniendo en cuenta las **frecuencias problemáticas**, es decir, los resultados de la **cuarta consulta**. Mientras que la parte de abajo está realizada con los resultados de las **primeras consultas**.

Apéndice IV

Tablas con R^2 por cada Compresor.

Comp_A	
Modelos	R2 Score
LinearRegression	0.949685
RandomForestRegressor	0.986857
AdaBoostRegressor	0.972466
GradientBoostingRegressor	0.983349
XGBRegressor	0.987457

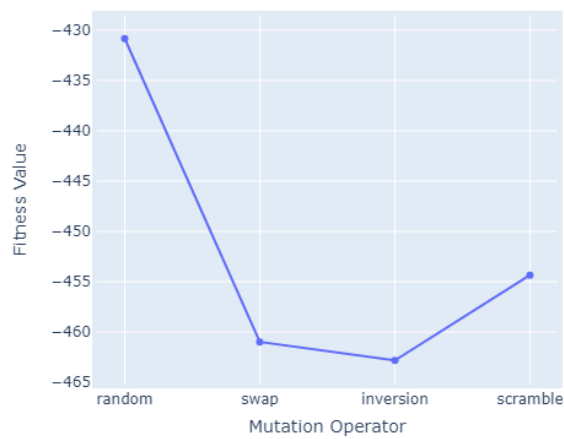
Comp_B	
Modelos	R2 Score
LinearRegression	0.962408
RandomForestRegressor	0.995552
AdaBoostRegressor	0.983092
GradientBoostingRegressor	0.993256
XGBRegressor	0.995042

Comp_C	
Modelos	R2 Score
LinearRegression	0.956558
RandomForestRegressor	0.994132
AdaBoostRegressor	0.981419
GradientBoostingRegressor	0.991364
XGBRegressor	0.993686

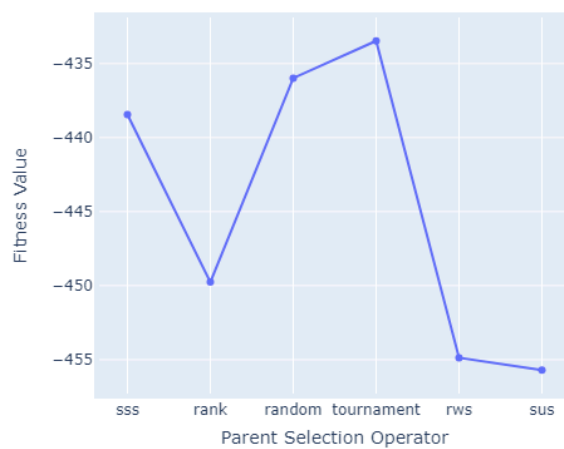
Comp_D	
Modelos	R2 Score
LinearRegression	0.962577
RandomForestRegressor	0.994633
AdaBoostRegressor	0.982395
GradientBoostingRegressor	0.992436
XGBRegressor	0.994376

Gráficos de comparación en cada parámetro

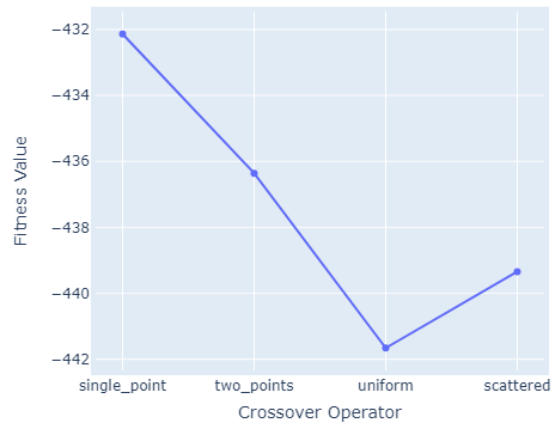
Comparación de los operadores de mutación



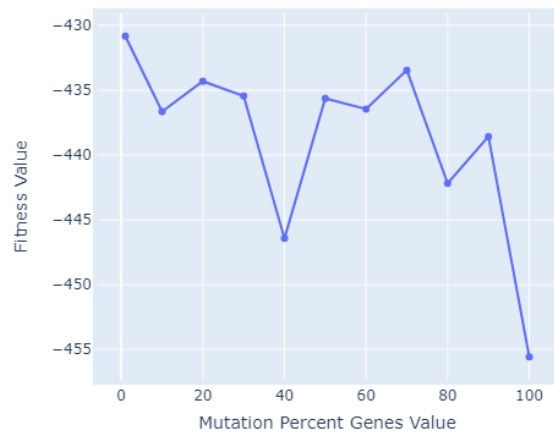
Comparación de los operadores de selección



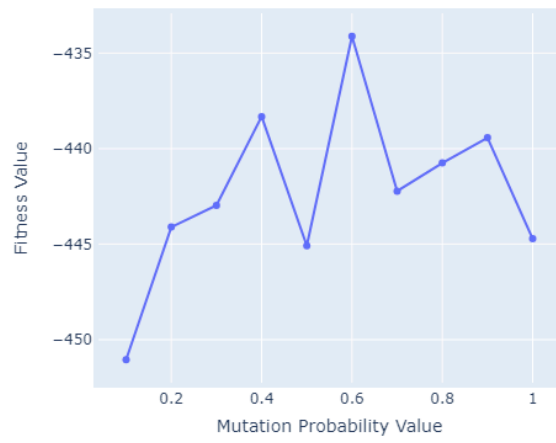
Comparación de los operadores de cruce



Comparación de los porcentajes de mutación



Comparación de las probabilidades de mutación



Glosarios

Glosario de siglas

Acrónimo	Significado
MQTT	Message Queuing Telemetry Transport
IoT	Internet of Things
SO	Sistema Operativo
SNMP	Simple Network Management Protocol
CSV	Comma Separated Values (tipo de archivo)
TXT	Text file (tipo archivo)

Glosario de términos

Big Data: Conjuntos de datos de gran tamaño y complejidad, provenientes de diversas fuentes. Su magnitud desafía el procesamiento convencional, pero ofrece oportunidades para resolver problemas empresariales previamente inabordables. (Oracle, 2024)

Stack Elastic: Conjunto de herramientas de código abierto para búsqueda, análisis y visualización de datos. Incluye Elasticsearch para almacenamiento y búsqueda, Logstash para procesamiento de datos, Kibana para visualización y Beats para la recolección de datos. (Elastic, 2024)

Máquina virtual: Entorno de computación simulado que opera como un sistema independiente dentro de otro sistema operativo, permitiendo la ejecución de software de manera aislada. (Ramírez, 2020)

Ubuntu: Sistema operativo de código abierto basado en Linux, conocido por su facilidad de uso y su comunidad activa. (V & V, 2023)

Mosquito: Broker MQTT de código abierto diseñado para facilitar la comunicación entre dispositivos IoT. Gestiona la suscripción y publicación de mensajes de manera eficiente y fiable. (Jecrespom, 2021)

Puerto origen y destino NAT: Los puertos origen y destino NAT se refieren a los puertos de origen y destino modificados durante el proceso de traducción de direcciones de red (NAT), que permite que múltiples dispositivos en una red privada compartan una única dirección IP pública para comunicarse con dispositivos fuera de la red privada.